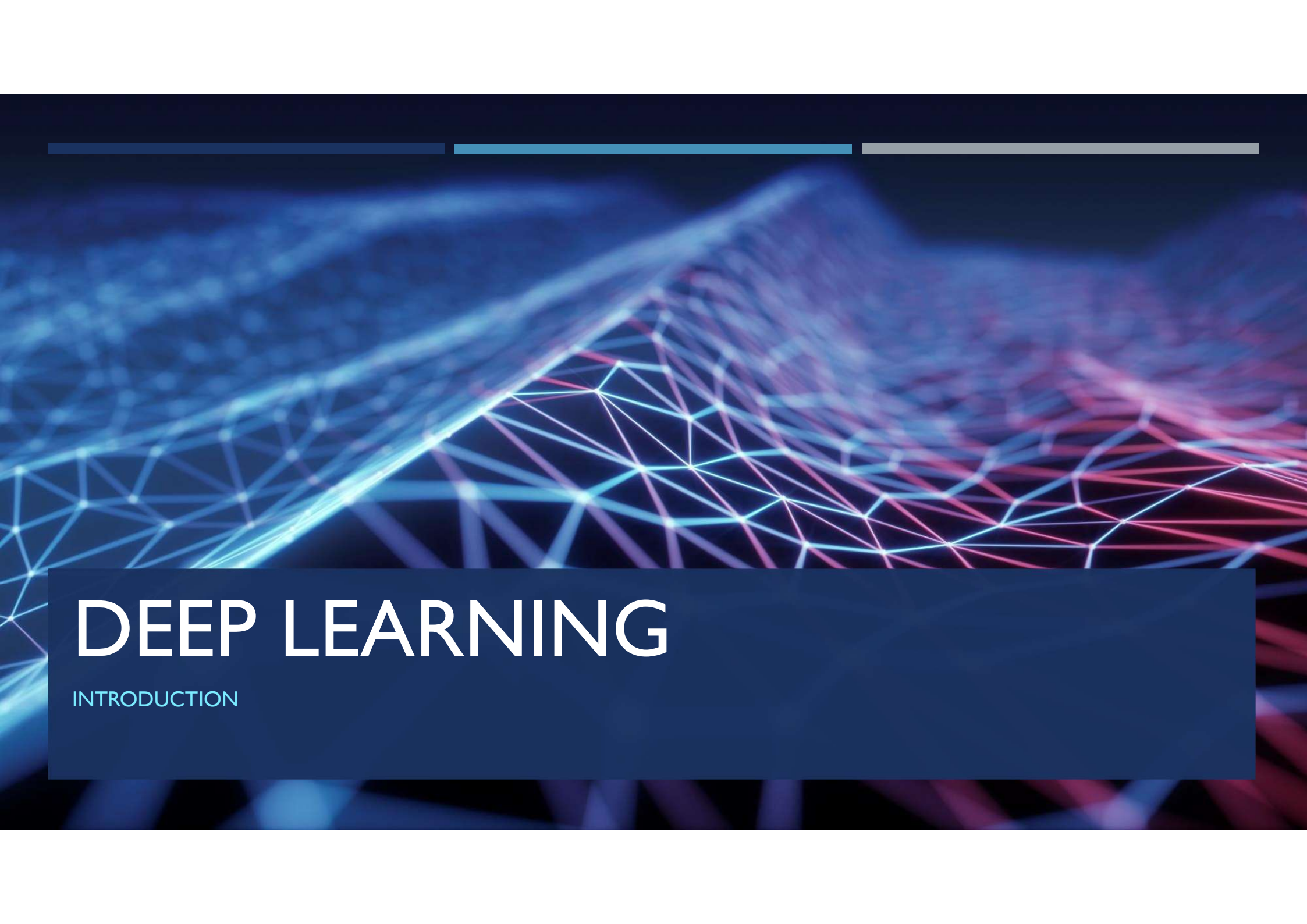
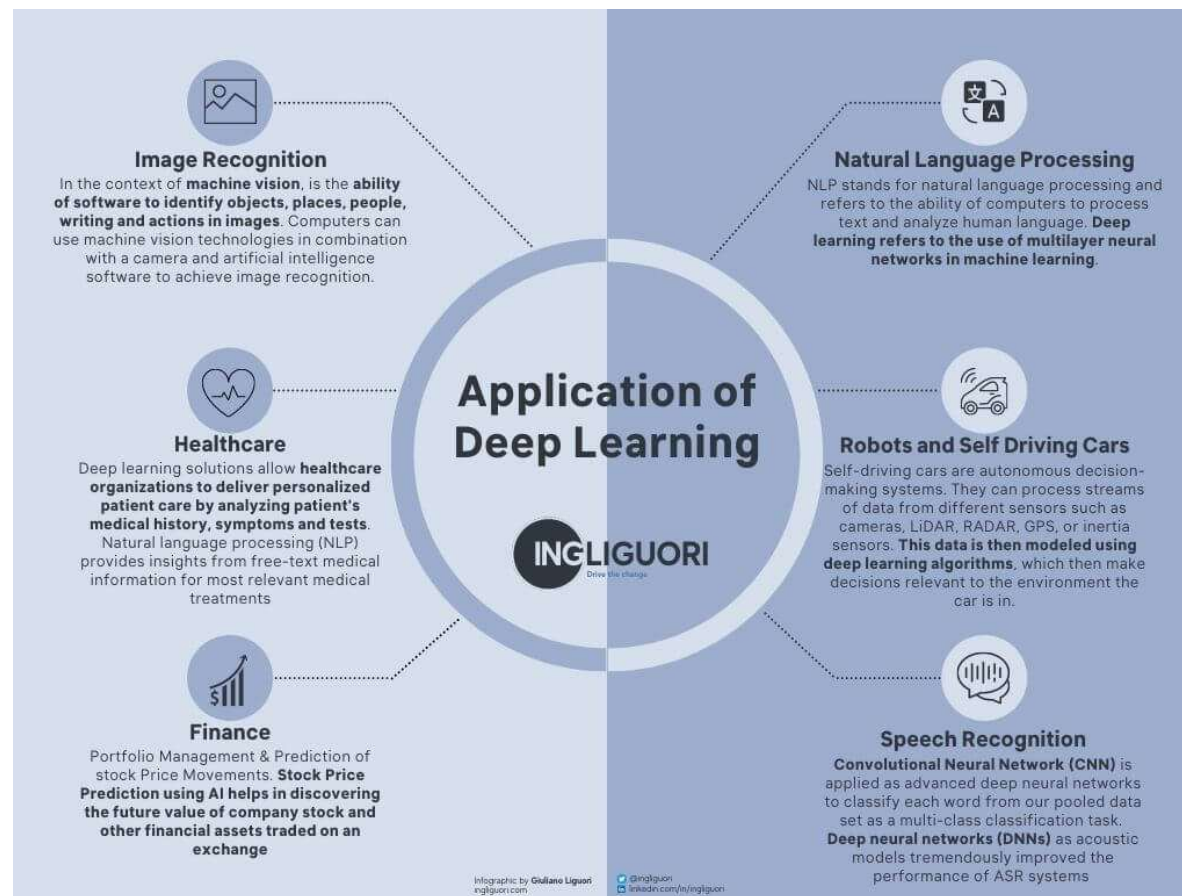




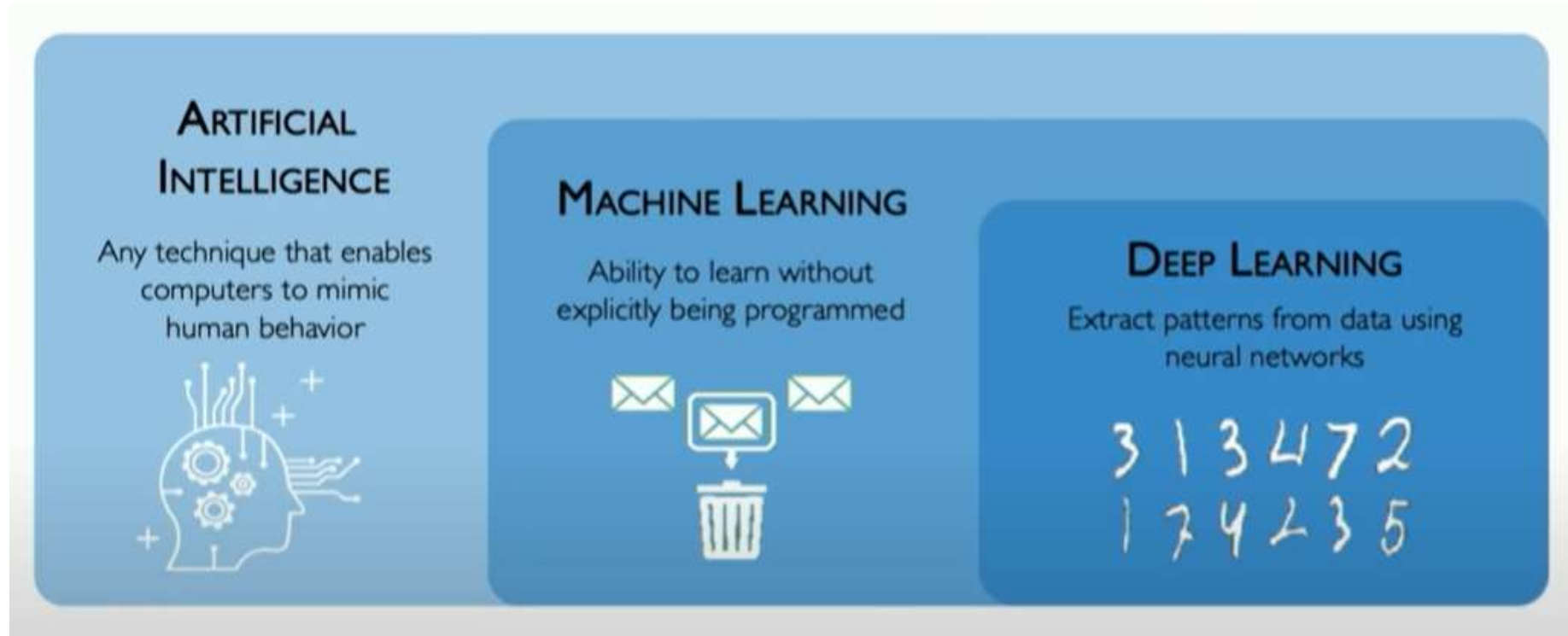
DEEP LEARNING

INTRODUCTION

APPLICATION OF DEEP LEARNING



WHAT IS DEEP LEARNING ?

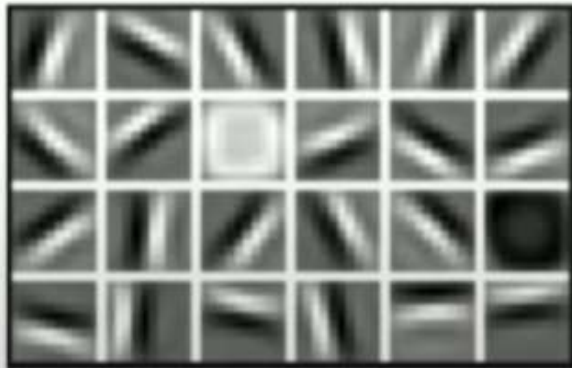


Teaching computer how to **learn a task** directly from **raw data**

WHY DEEP LEARNING

Hand engineered features are time consuming, brittle, and not scalable in practice
Can we learn the **underlying features** directly from data?

Low Level Features



Lines & Edges

Mid Level Features



Eyes & Nose & Ears

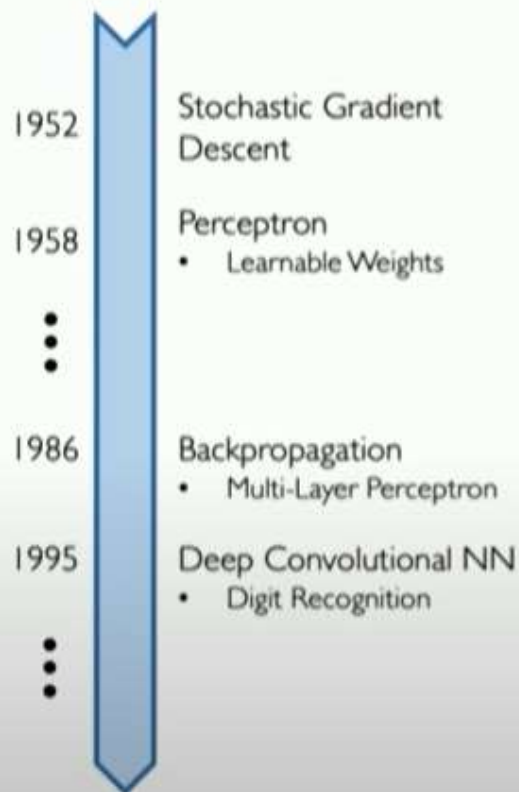
High Level Features



Facial Structure

WHY NOW

Neural Networks date back decades, so why the resurgence?



1. Big Data

- Larger Datasets
- Easier Collection & Storage

IMAGENET



WIKIPEDIA
The Free Encyclopedia



2. Hardware

- Graphics Processing Units (GPUs)
- Massively Parallelizable

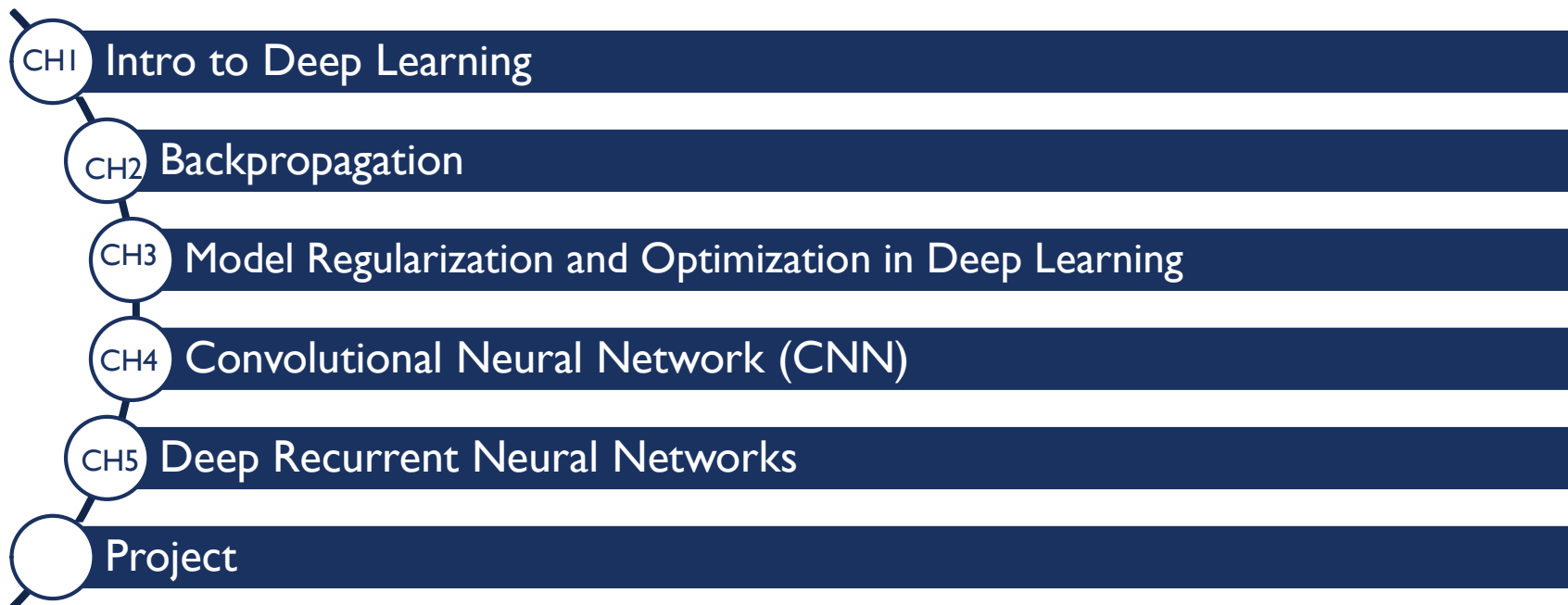


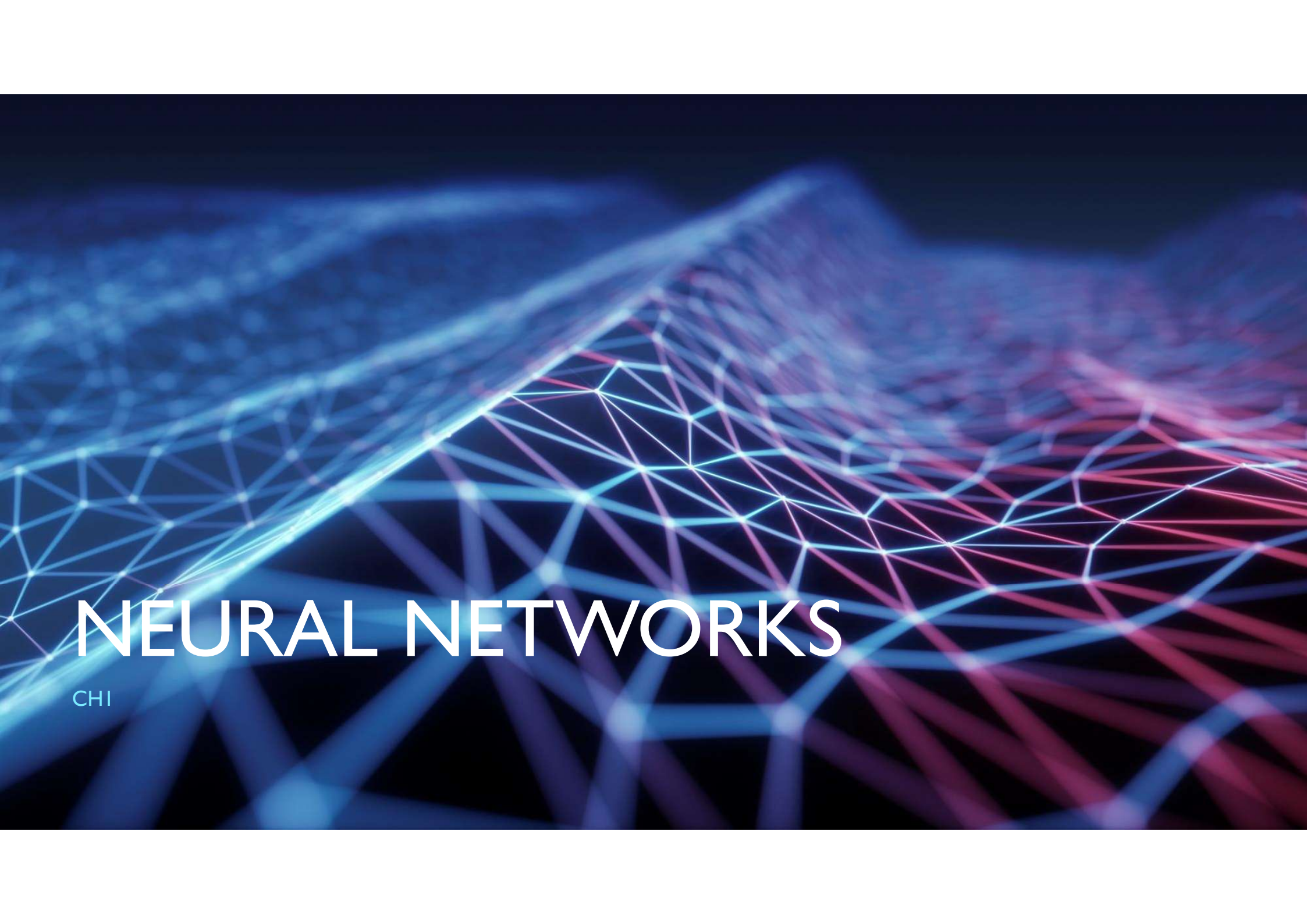
3. Software

- Improved Techniques
- New Models
- Toolboxes



LECTURE SCHEDULE





NEURAL NETWORKS

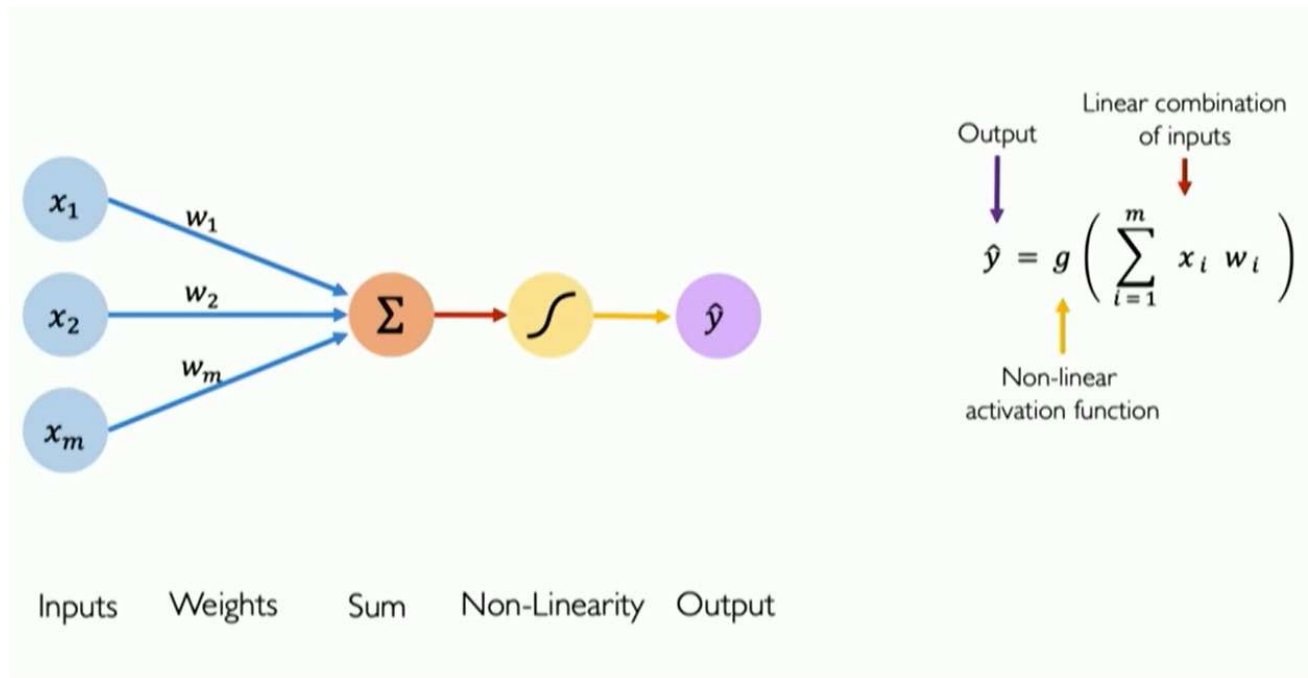
CHI

THE PERCEPTRON

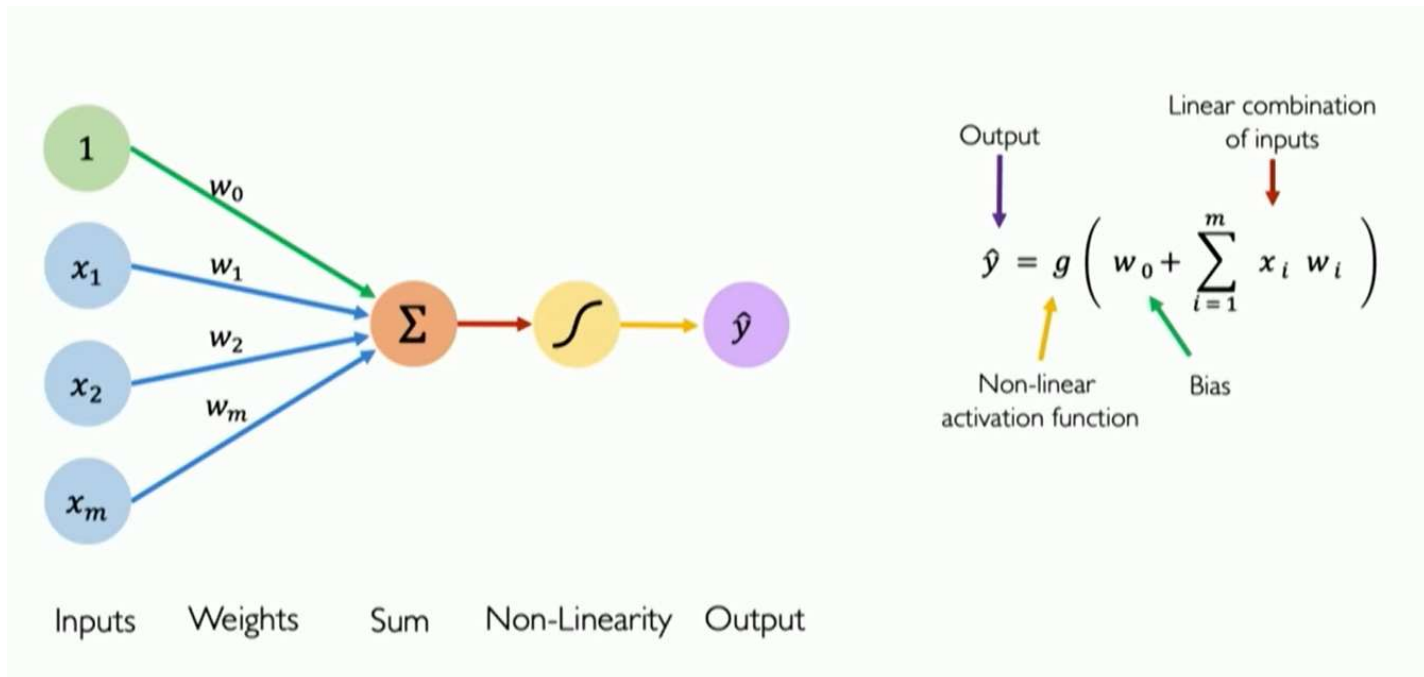
THE STRUCTURAL BUILDING BLOCK OF DEEP LEARNING



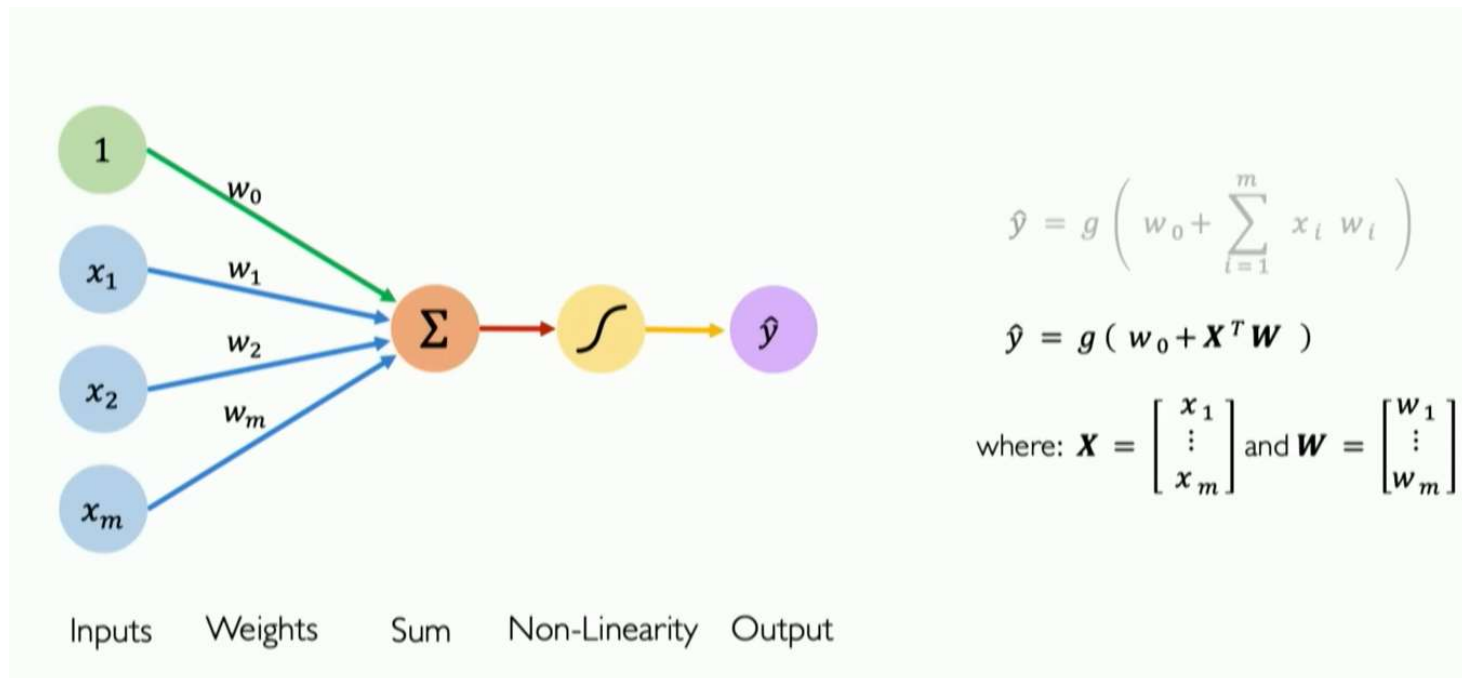
THE PERCEPTRON : FORWARD PROPAGATION



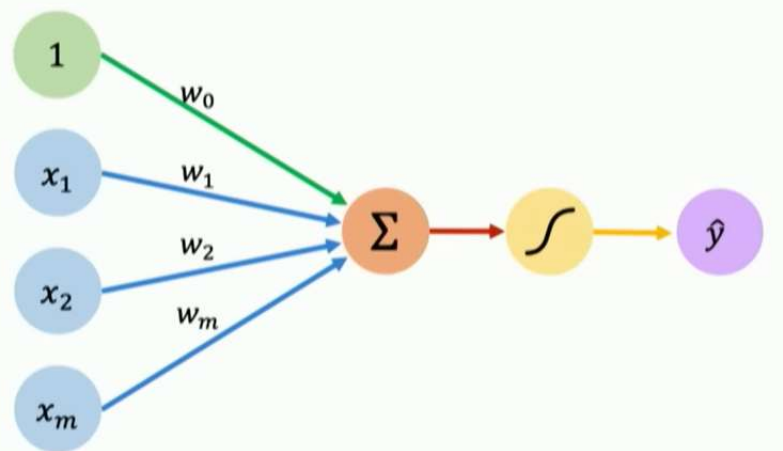
THE PERCEPTRON : FORWARD PROPAGATION



THE PERCEPTRON : FORWARD PROPAGATION



THE PERCEPTRON : FORWARD PROPAGATION



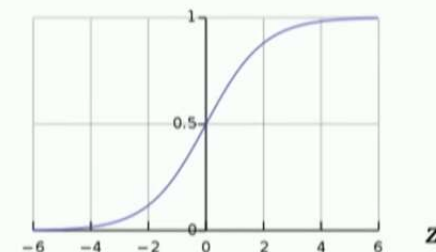
Inputs Weights Sum Non-Linearity Output

Activation Functions

$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

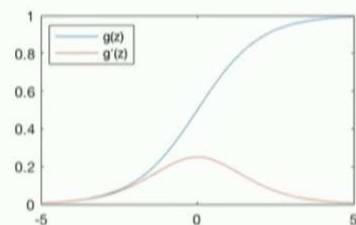
- Example: sigmoid function

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



COMMON ACTIVATION FUNCTIONS

Sigmoid Function

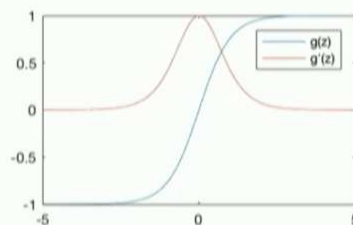


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

 `tf.math.sigmoid(z)`

Hyperbolic Tangent

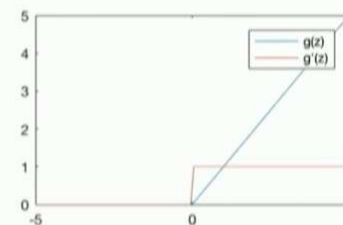


$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

 `tf.math.tanh(z)`

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

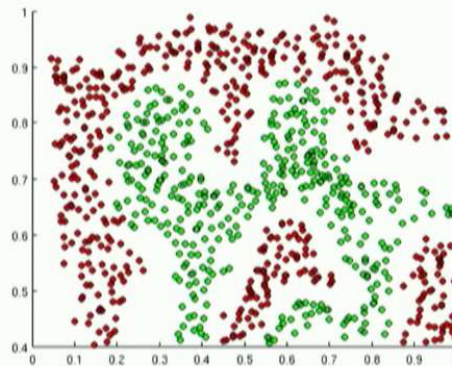
 `tf.nn.relu(z)`

 TensorFlow code blocks

NOTE: All activation functions are non-linear

IMPORTANCE OF ACTIVATION FUNCTIONS

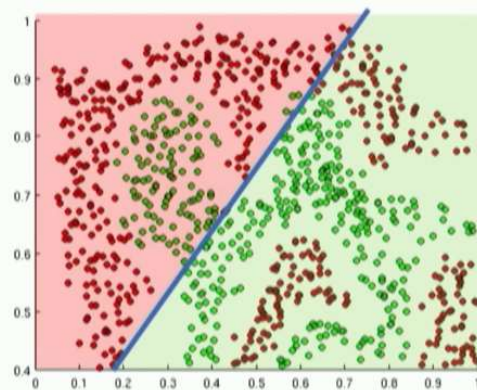
The purpose of activation functions is to **introduce non-linearities** into the network



What if we wanted to build a neural network to distinguish green vs red points?

IMPORTANCE OF ACTIVATION FUNCTIONS

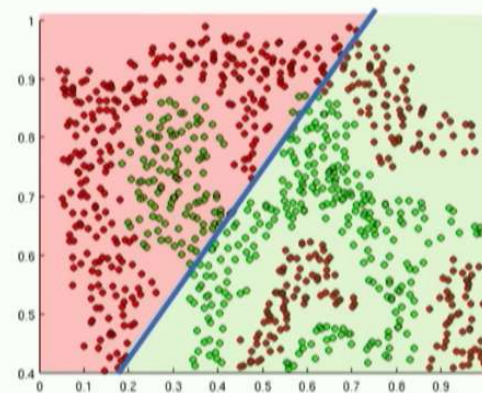
The purpose of activation functions is to **introduce non-linearities** into the network



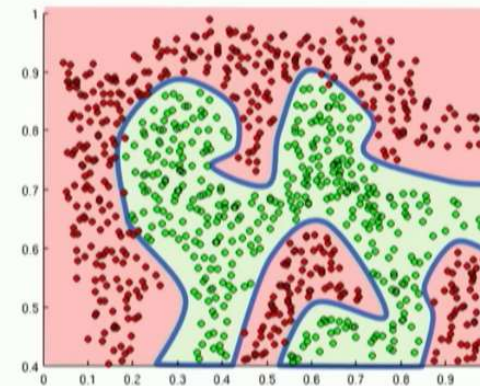
Linear activation functions produce linear decisions no matter the network size

IMPORTANCE OF ACTIVATION FUNCTIONS

The purpose of activation functions is to **introduce non-linearities** into the network

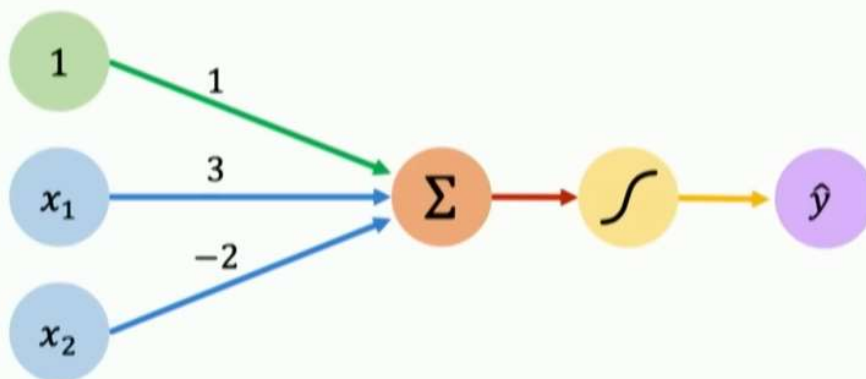


Linear activation functions produce linear decisions no matter the network size



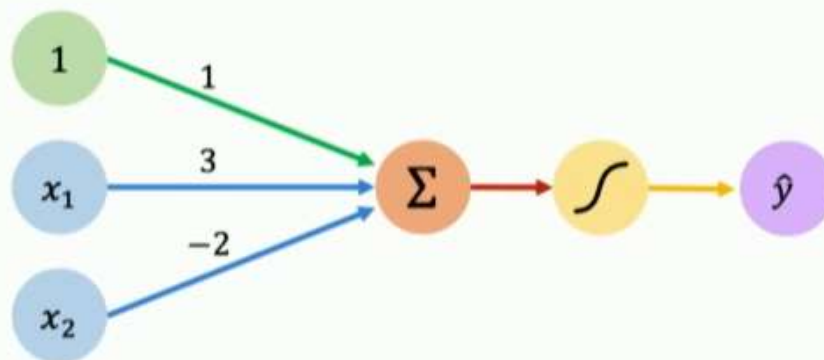
Non-linearities allow us to approximate arbitrarily complex functions

THE PERCEPTRON: EXAMPLE



We have: $w_0 = 1$ and $\mathbf{w} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$

THE PERCEPTRON: EXAMPLE

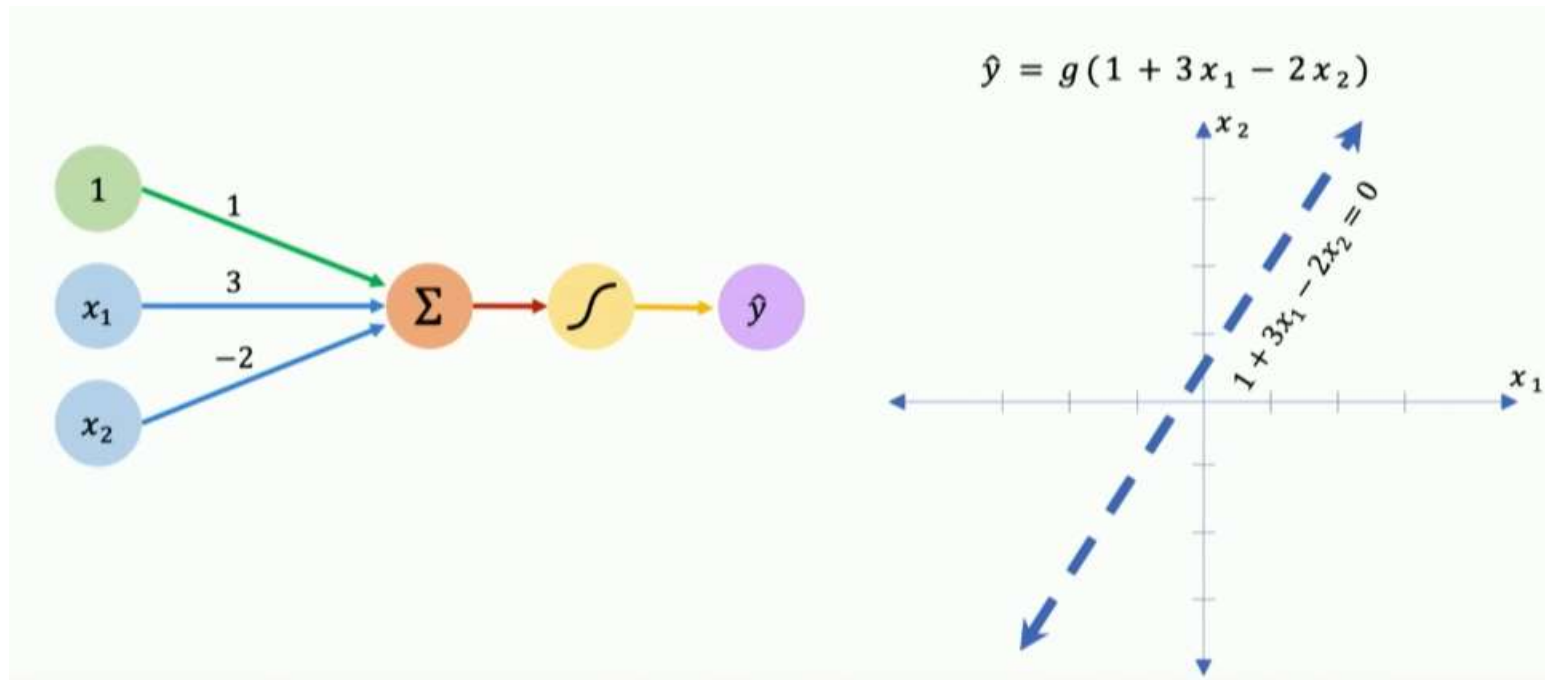


We have: $w_0 = 1$ and $\mathbf{w} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$

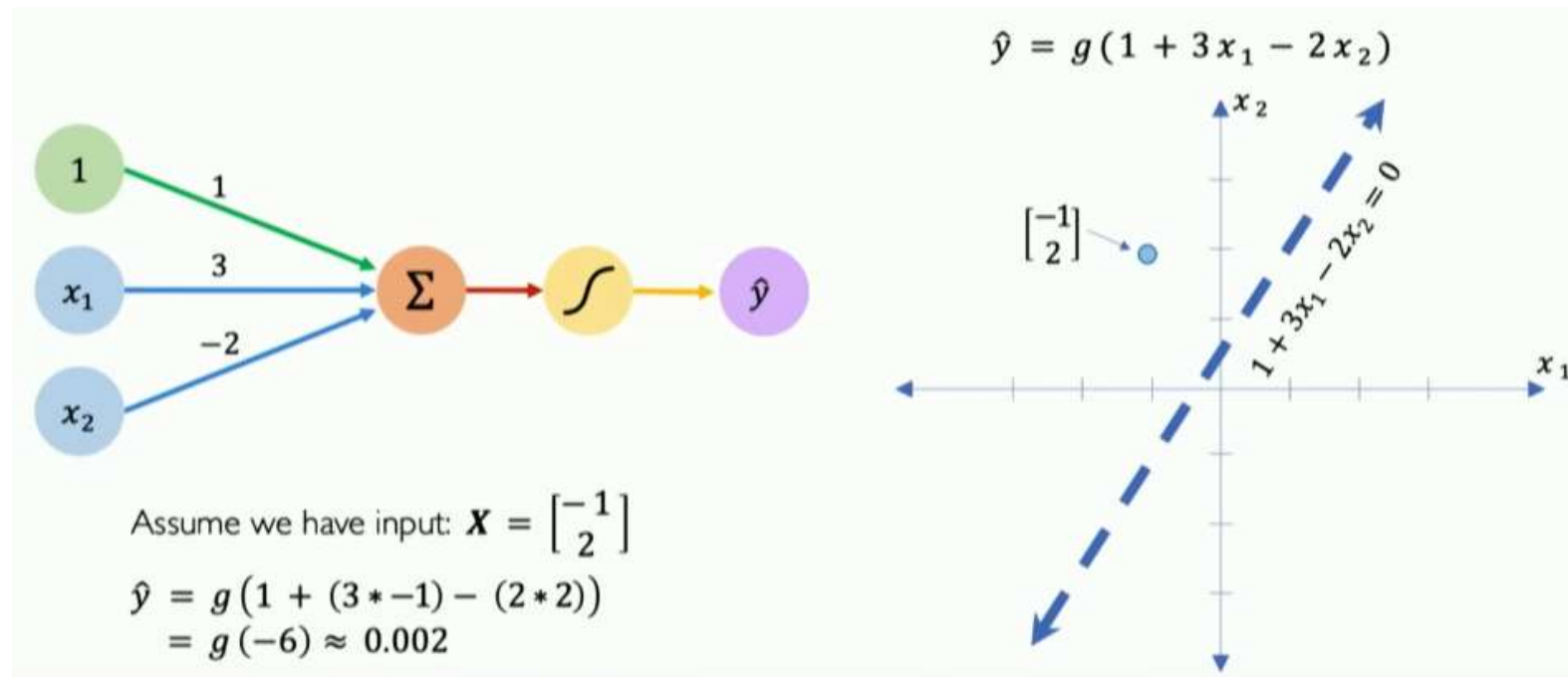
$$\begin{aligned}\hat{y} &= g(w_0 + \mathbf{X}^T \mathbf{w}) \\ &= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right) \\ \hat{y} &= g(1 + 3x_1 - 2x_2)\end{aligned}$$

This is just a line in 2D!

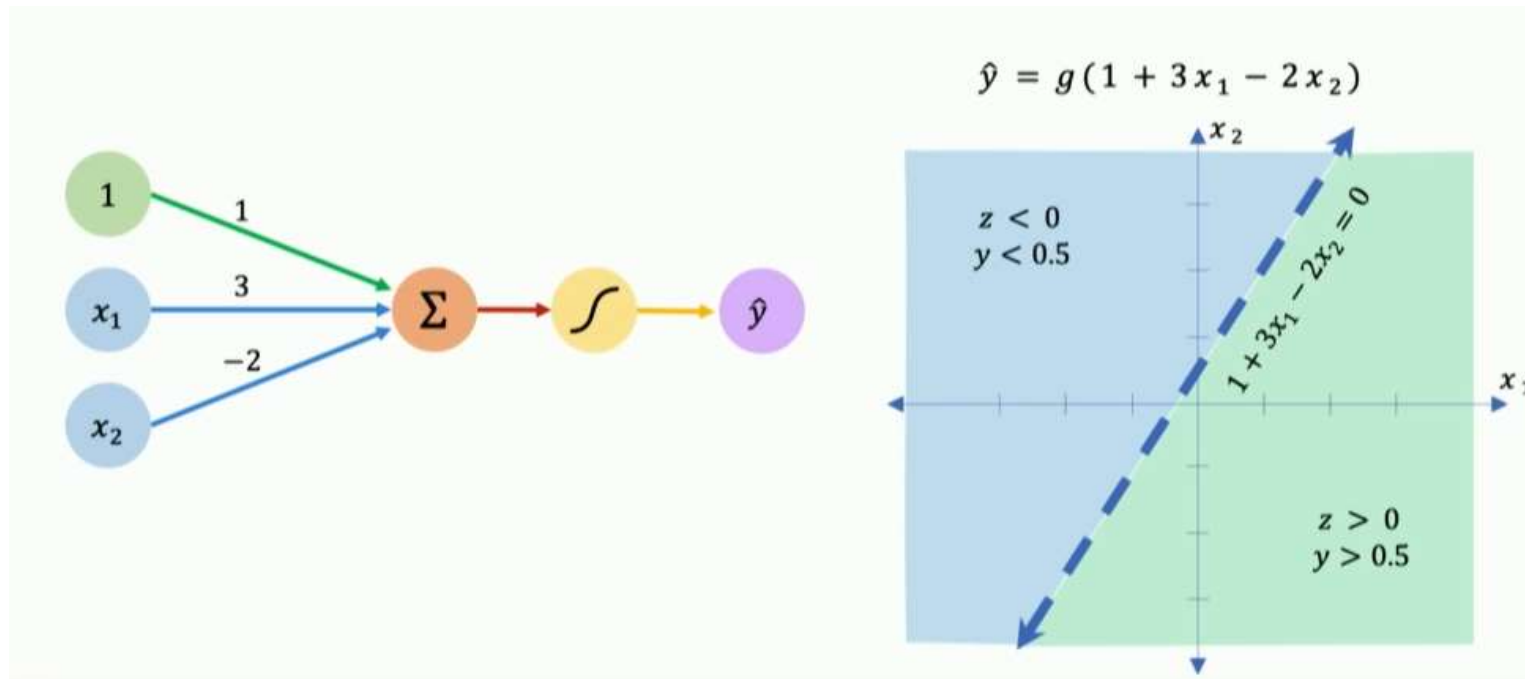
THE PERCEPTRON: EXAMPLE



THE PERCEPTRON: EXAMPLE



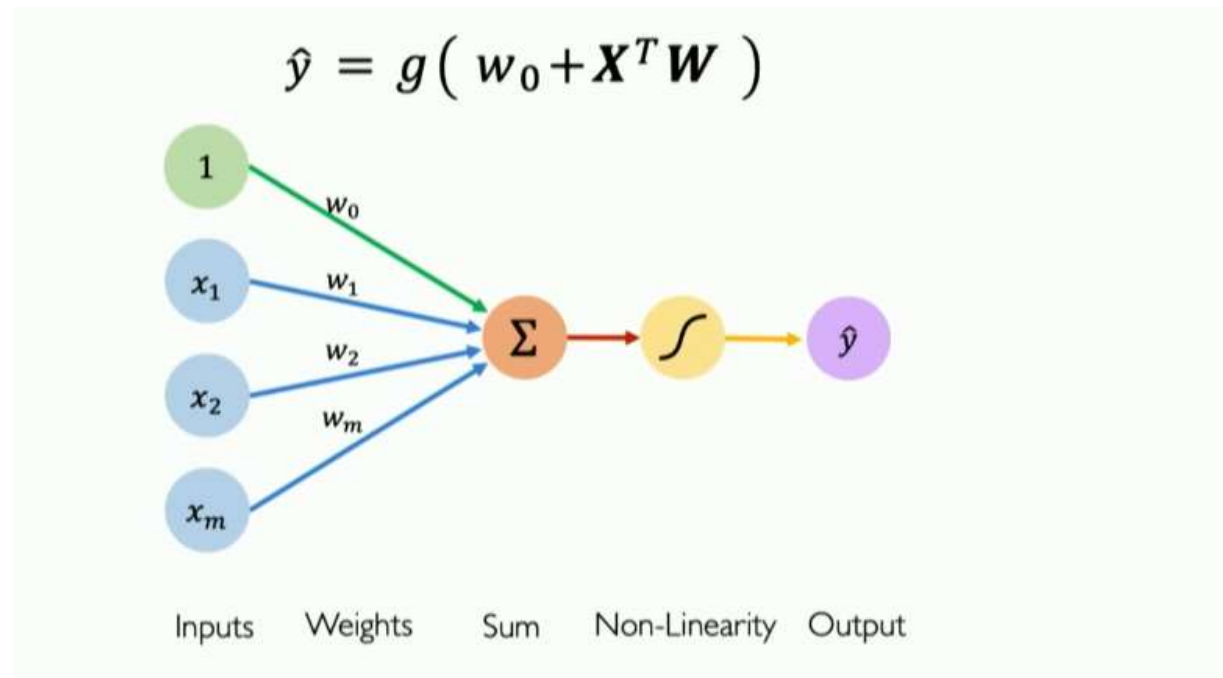
THE PERCEPTRON: EXAMPLE



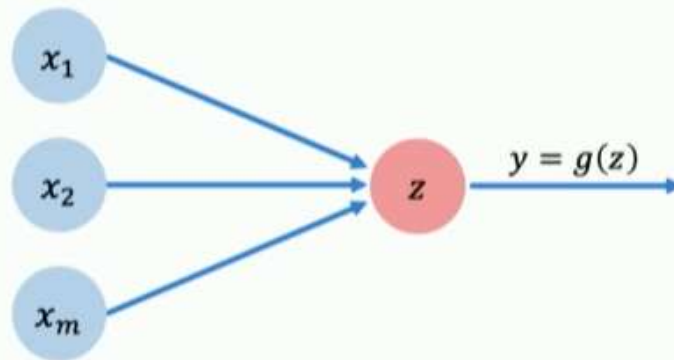
BUILDING NEURAL NETWORKS WITH PERCEPTRONS



THE PERCEPTRON SIMPLIFIED



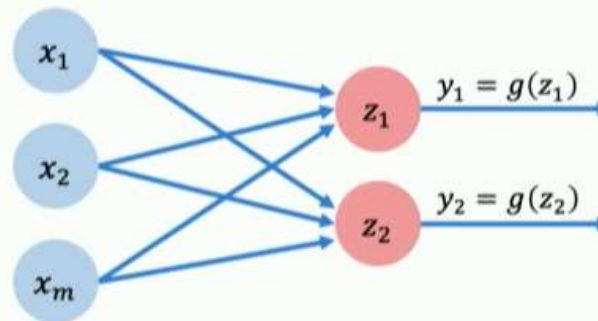
THE PERCEPTRON SIMPLIFIED



$$z = w_0 + \sum_{j=1}^m x_j w_j$$

MULTI OUTPUT PERCEPTRON

Because all inputs are densely connected to all outputs, these layers are called **Dense** layers



$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

DANSE LAYSER FROM SCRATCH

```
class MyDenseLayer(tf.keras.layers.Layer):
    def __init__(self, input_dim, output_dim):
        super(MyDenseLayer, self).__init__()

        # Initialize weights and bias
        self.W = self.add_weight([input_dim, output_dim])
        self.b = self.add_weight([1, output_dim])

    def call(self, inputs):
        # Forward propagate the inputs
        z = tf.matmul(inputs, self.W) + self.b

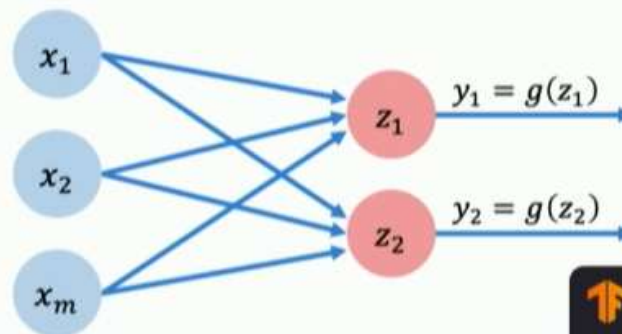
        # Feed through a non-linear activation
        output = tf.math.sigmoid(z)

        return output
```



MULTI OUTPUT PERCEPTRON

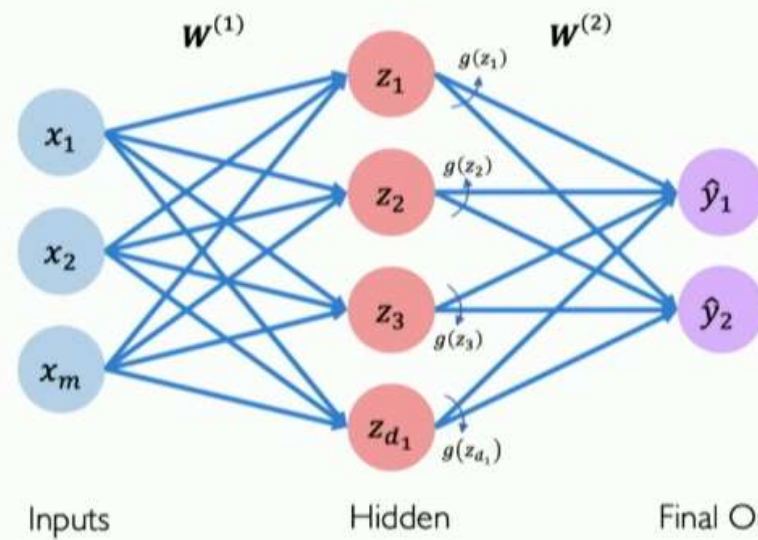
Because all inputs are densely connected to all outputs, these layers are called **Dense** layers



```
import tensorflow as tf  
layer = tf.keras.layers.Dense(  
    units=2)
```

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

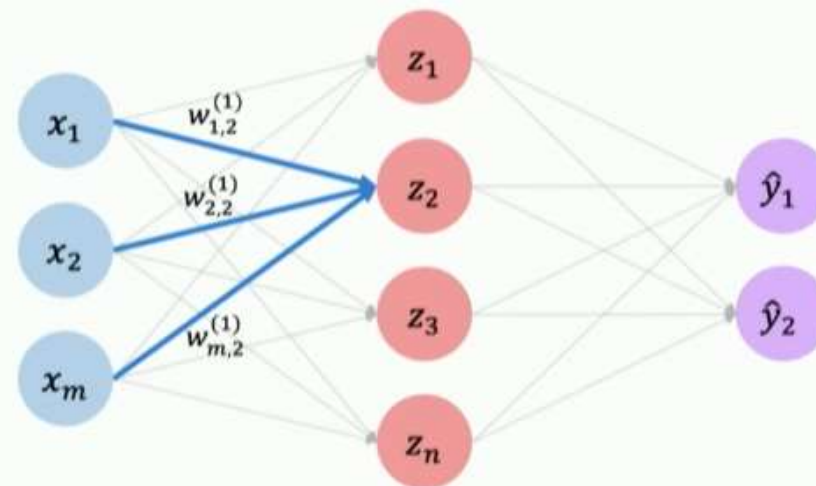
SINGLE LAYER NEURAL NETWORK



$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)}$$

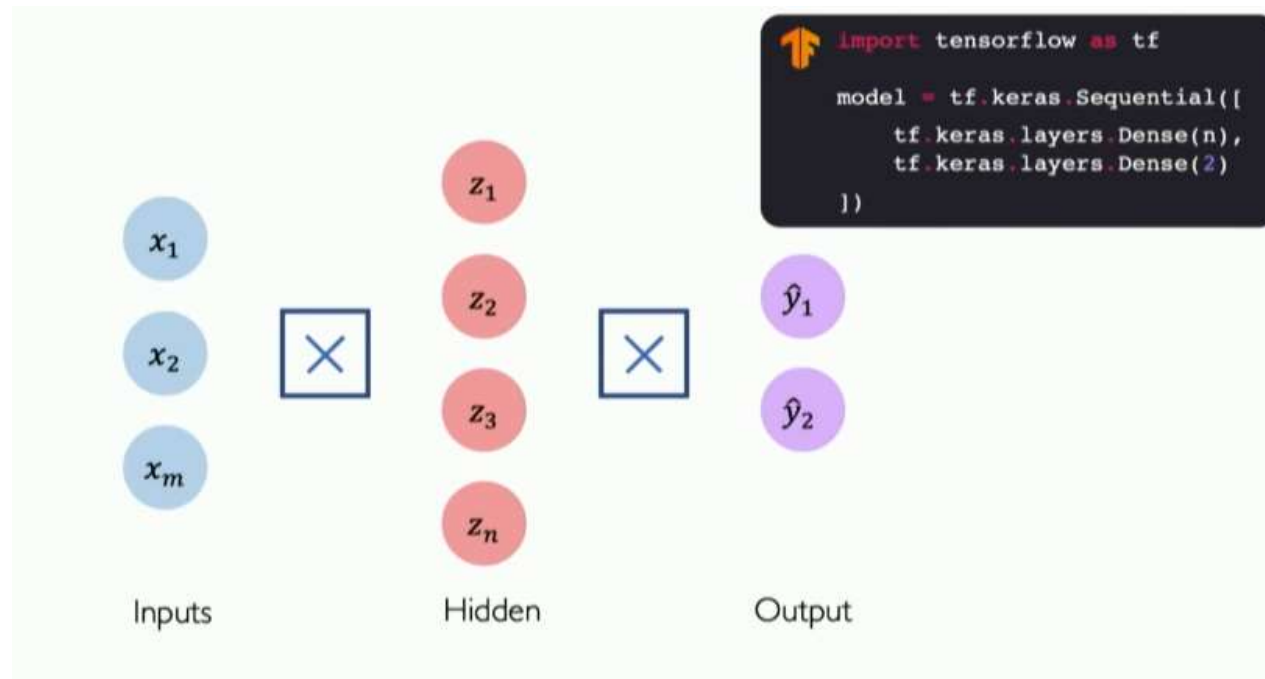
$$\hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,i}^{(2)} \right)$$

SINGLE LAYER NEURAL NETWORK

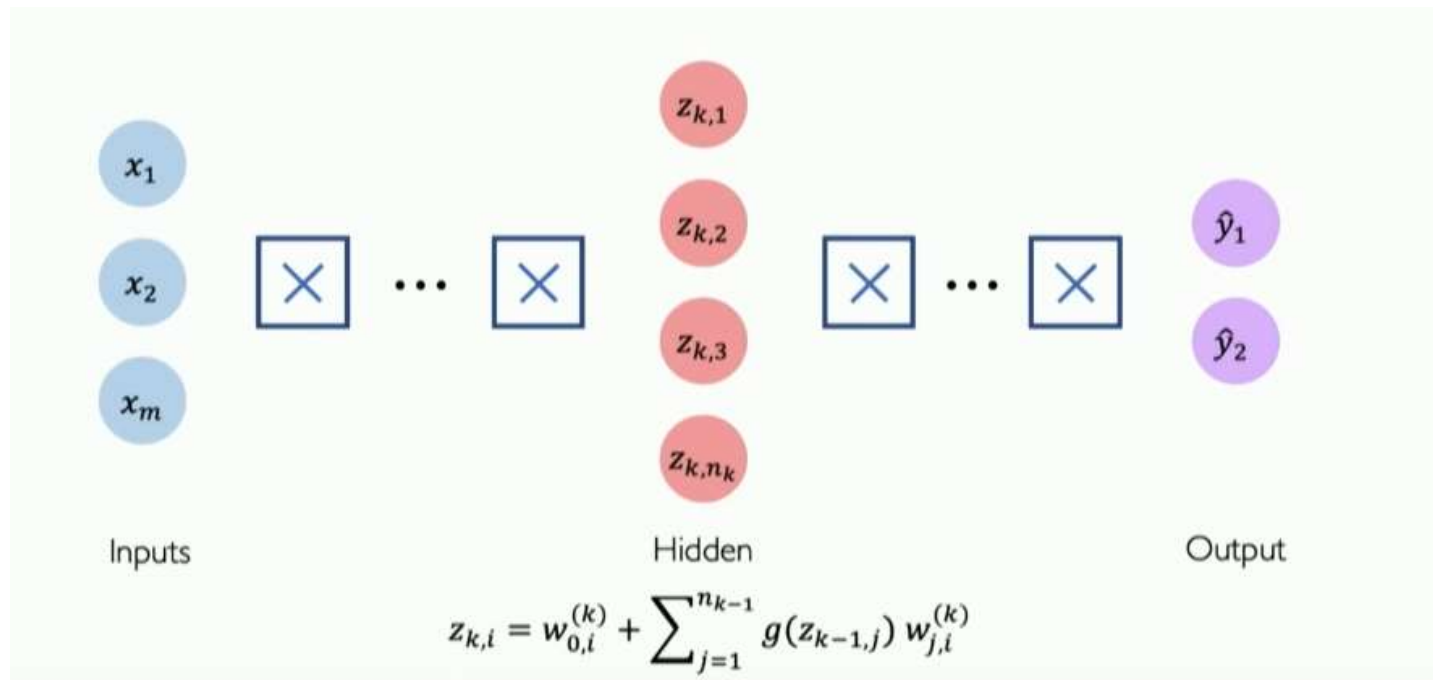


$$\begin{aligned} z_2 &= w_{0,2}^{(1)} + \sum_{j=1}^m x_j w_{j,2}^{(1)} \\ &= w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_m w_{m,2}^{(1)} \end{aligned}$$

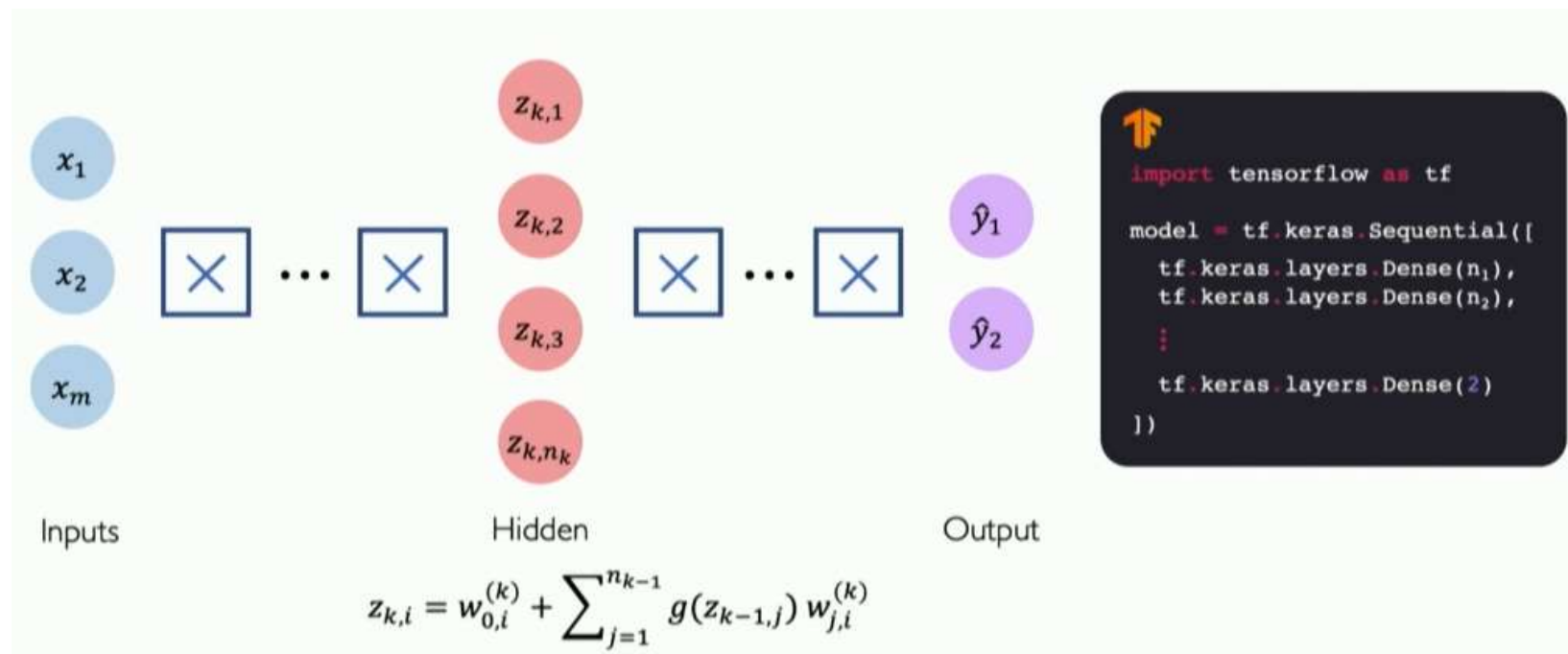
MULTI OUTPUT PERCEPTRON



DEEP NEURAL NETWORK



DEEP NEURAL NETWORKS





APPLYING NEURAL NETWORKS



EXAMPLE PROBLEM

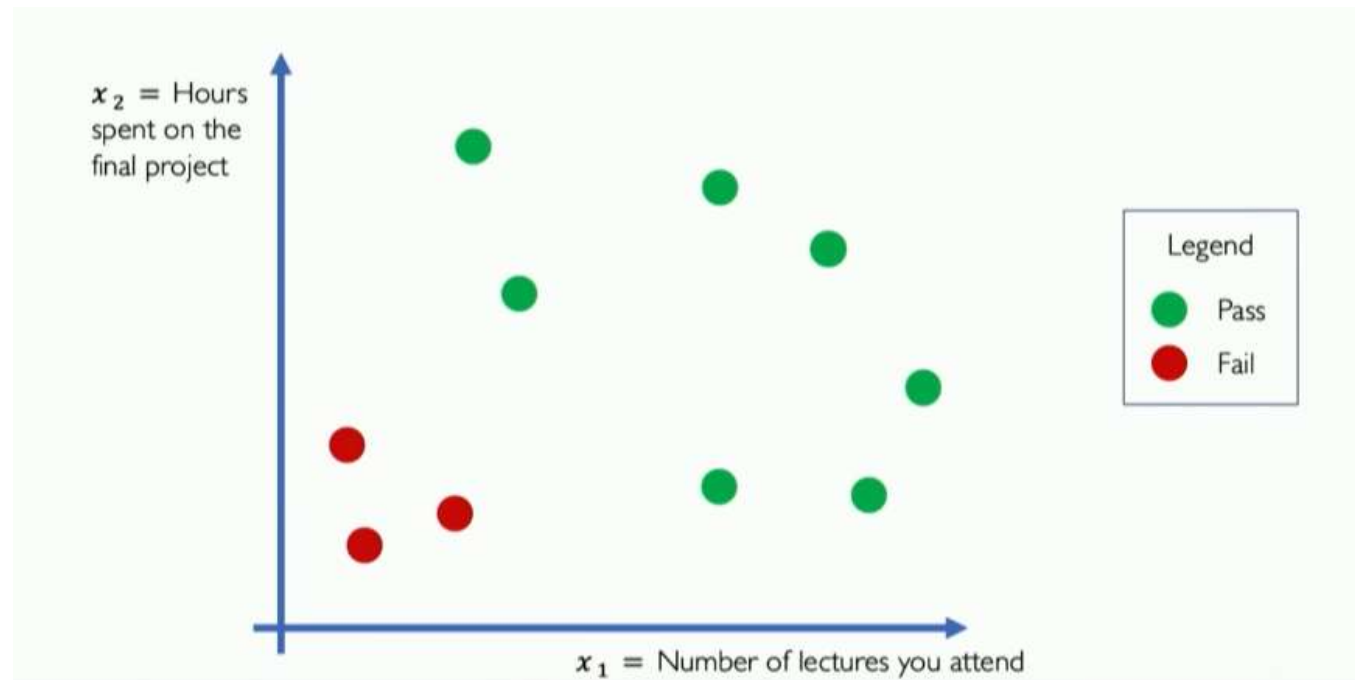
Will I pass this class?

Let's start with a simple two feature model

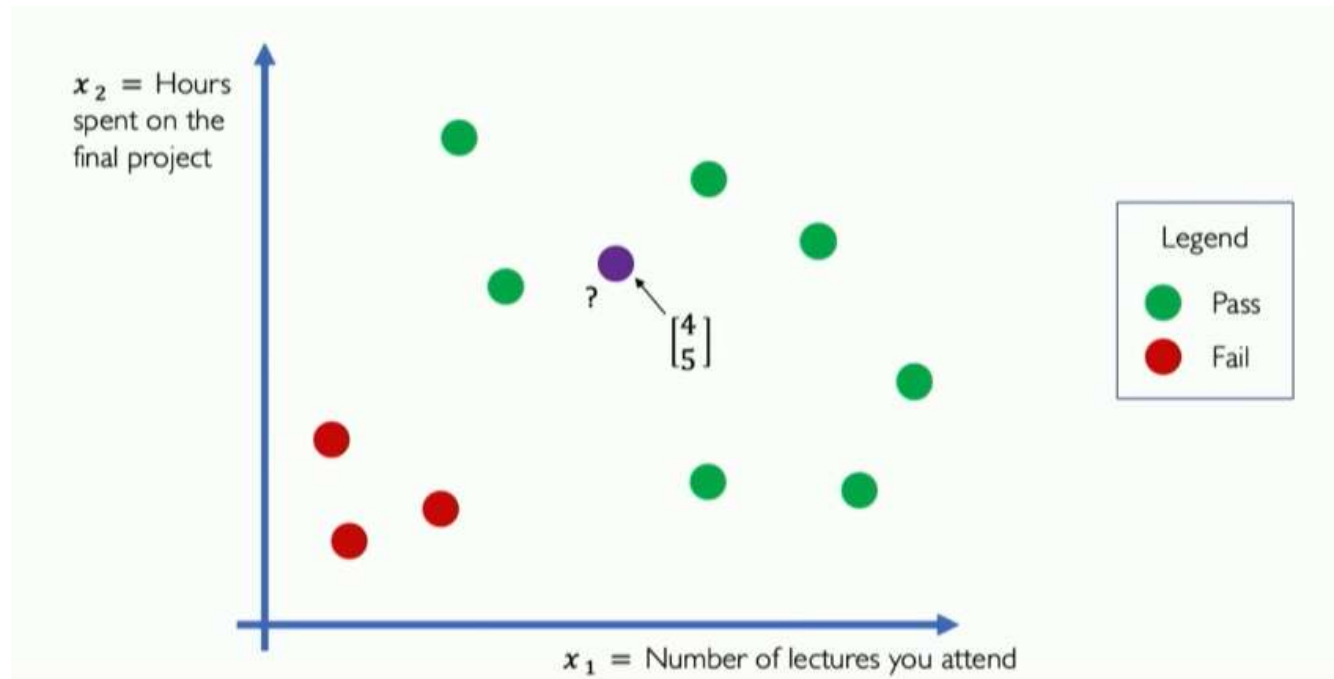
x_1 = Number of lectures you attend

x_2 = Hours spent on the final project

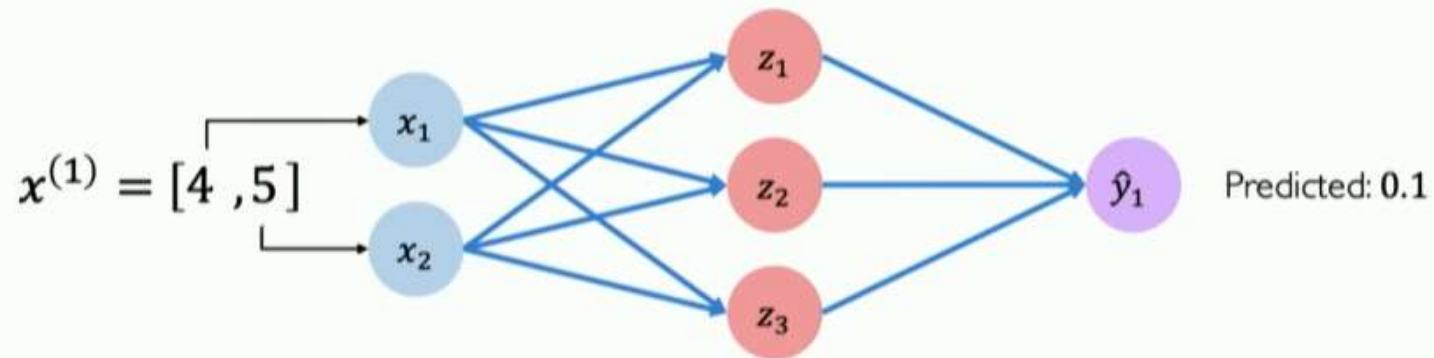
EXAMPLE PROBLEM : WILL I PASS THIS CLASS ?



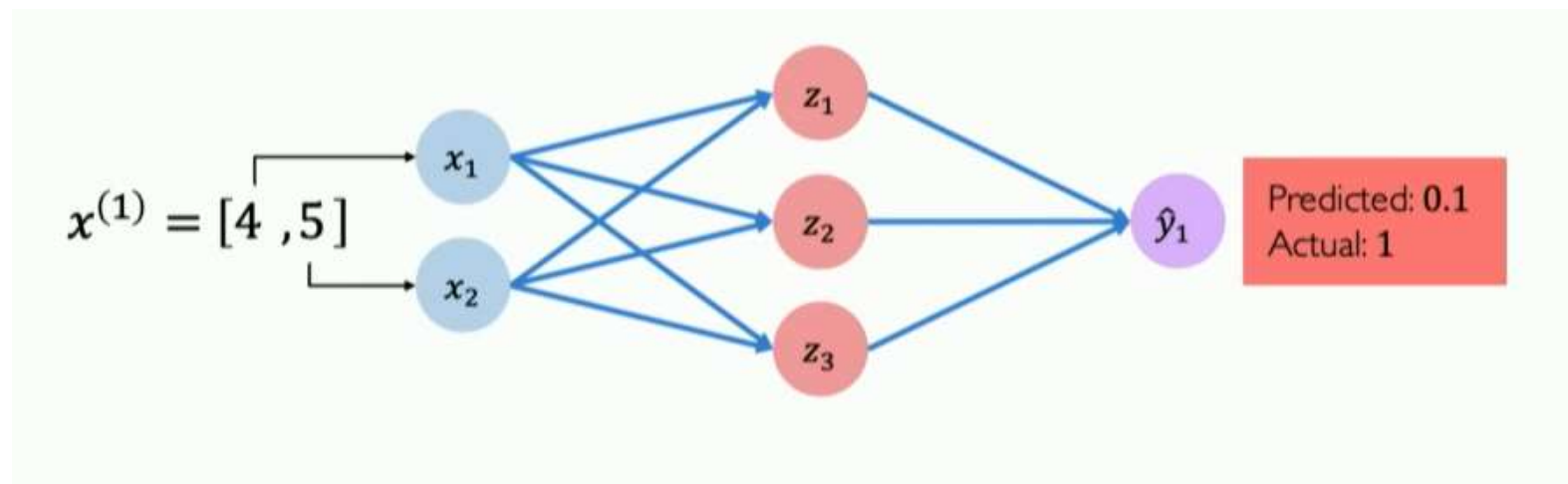
EXAMPLE PROBLEM : WILL I PASS THIS CLASS ?



EXAMPLE PROBLEM : WILL I PASS THIS CLASS ?

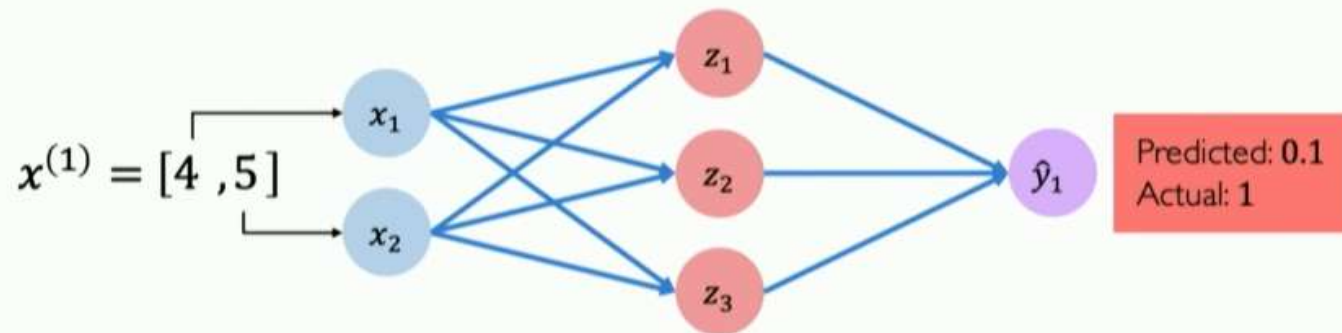


EXAMPLE PROBLEM : WILL I PASS THIS CLASS ?



QUANTIFYING LOSS

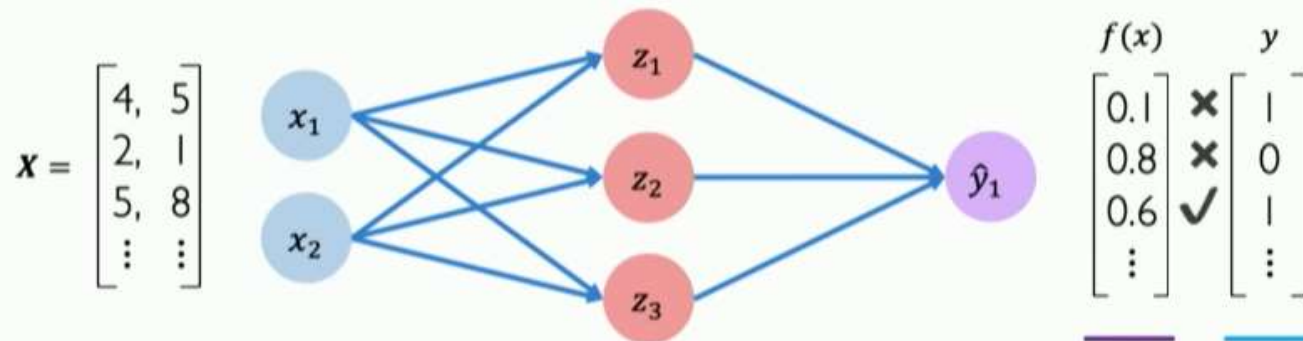
The **loss** of our network measures the cost incurred from incorrect predictions



$$\mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

EMPIRICAL LOSS

The **empirical loss** measures the total loss over our entire dataset

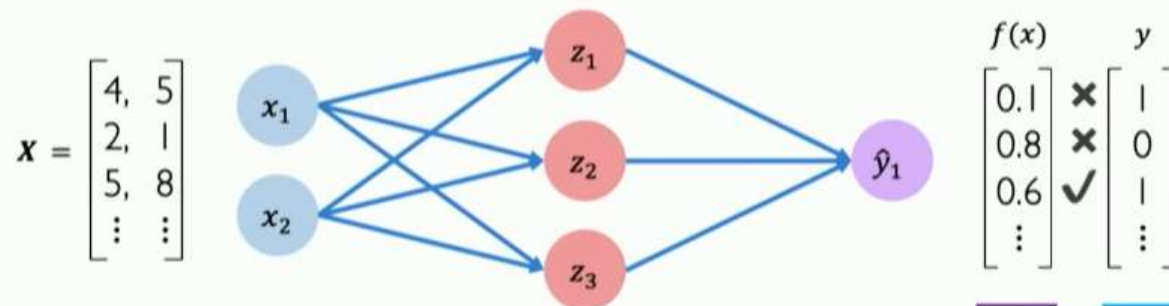


- Also known as:
- Objective function
 - Cost function
 - Empirical Risk

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

BINARY CROSS ENTROPY LOSS

Cross entropy loss can be used with models that output a probability between 0 and 1



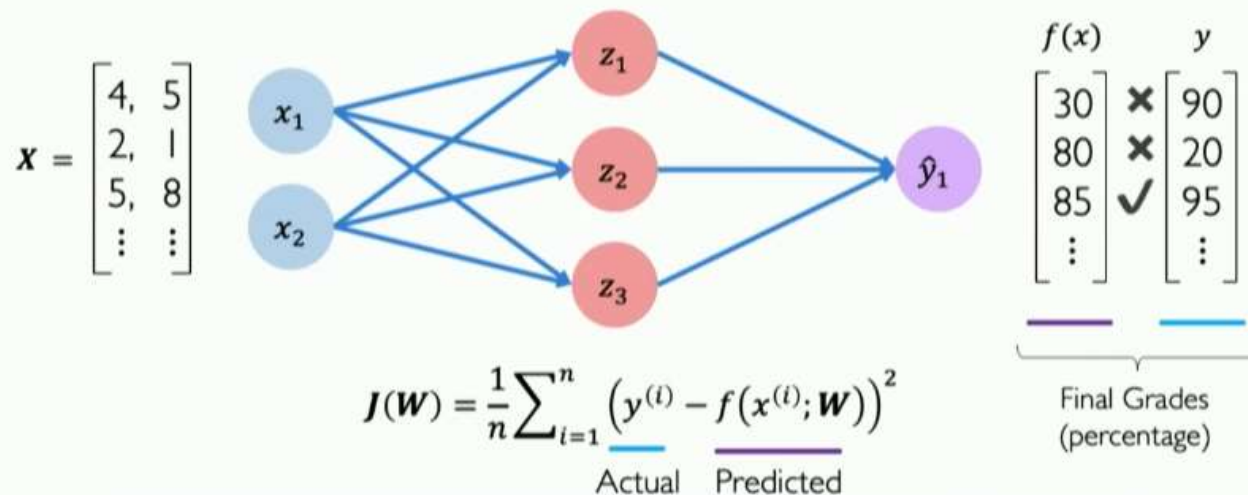
$$J(W) = -\frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Actual}} \log \left(\underbrace{f(x^{(i)}; W)}_{\text{Predicted}} \right) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log \left(1 - \underbrace{f(x^{(i)}; W)}_{\text{Predicted}} \right)$$



```
loss = tf.reduce_mean( tf.nn.softmax_cross_entropy_with_logits(y, predicted) )
```

MEAN SQUARED ERROR LOSS

Mean squared error loss can be used with regression models that output continuous real numbers



```
loss = tf.reduce_mean( tf.square(tf.subtract(y, predicted)) )  
loss = tf.keras.losses.MSE( y, predicted )
```



TRAINING NEURAL NETWORKS



LOSS OPTIMIZATION

We want to find the network weights that **achieve the lowest loss**

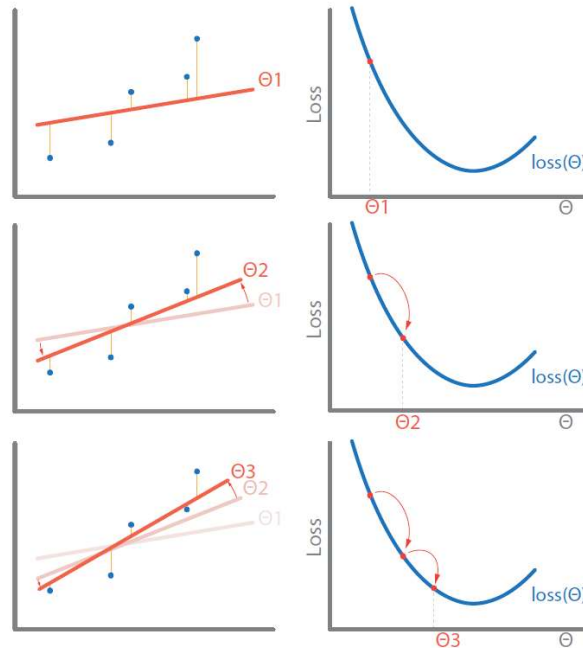
$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

LOSS OPTIMIZATION : LINEAR REGRESSION

We are looking for a straight line that passes « as close as possible » to our points.

We search $\hat{\theta} = \begin{pmatrix} \hat{a} \\ \hat{b} \end{pmatrix}$ where $\hat{y} = \hat{a} \cdot x + \hat{b}$ will be a prediction



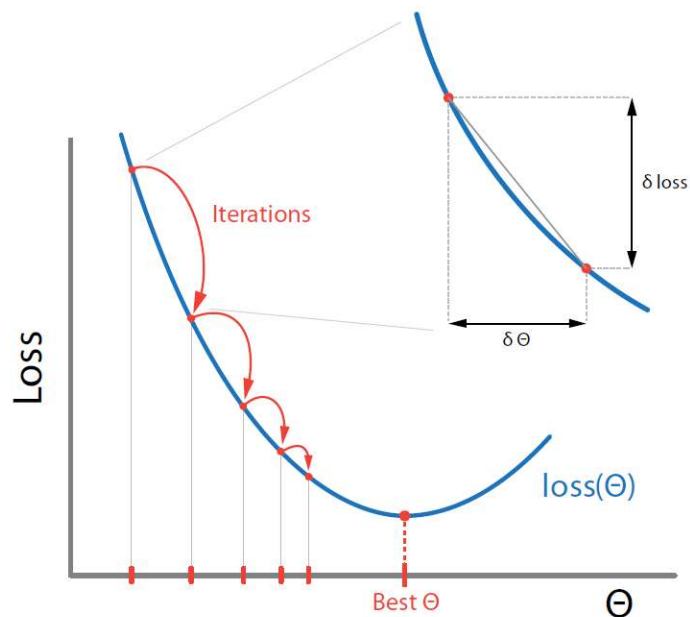
↻ We will iteratively look for the best position of our line, by varying its parameters (Θ).

? But how can we efficiently vary our parameters (Θ)?

Note : Loss functions could be :

$$\text{RMSE}(X, h_{\theta}) = \sqrt{\frac{1}{n} \sum_{i=1}^n [h_{\theta}(X^{(i)}) - Y^{(i)}]^2}$$
$$\text{MSE}(X, h_{\theta}) = \frac{1}{n} \sum_{i=1}^n [h_{\theta}(X^{(i)}) - Y^{(i)}]^2$$

GRADIENT DESCENT : LINEAR REGRESSION



By changing Θ from $\delta\Theta$
We improve $loss(\Theta)$ of $\delta loss$

The gradient is the slope we will follow
to minimize our loss function.

$$\text{gradient} = \frac{\delta loss}{\delta \theta}$$

$$\text{One iterative solution is : } \theta \leftarrow \theta - \eta \cdot \frac{\delta loss}{\delta \theta}$$

where η is the learning rate

This process is called **gradient descent** and
the function used to optimize the descent,
optimization function

LOSS OPTIMIZATION

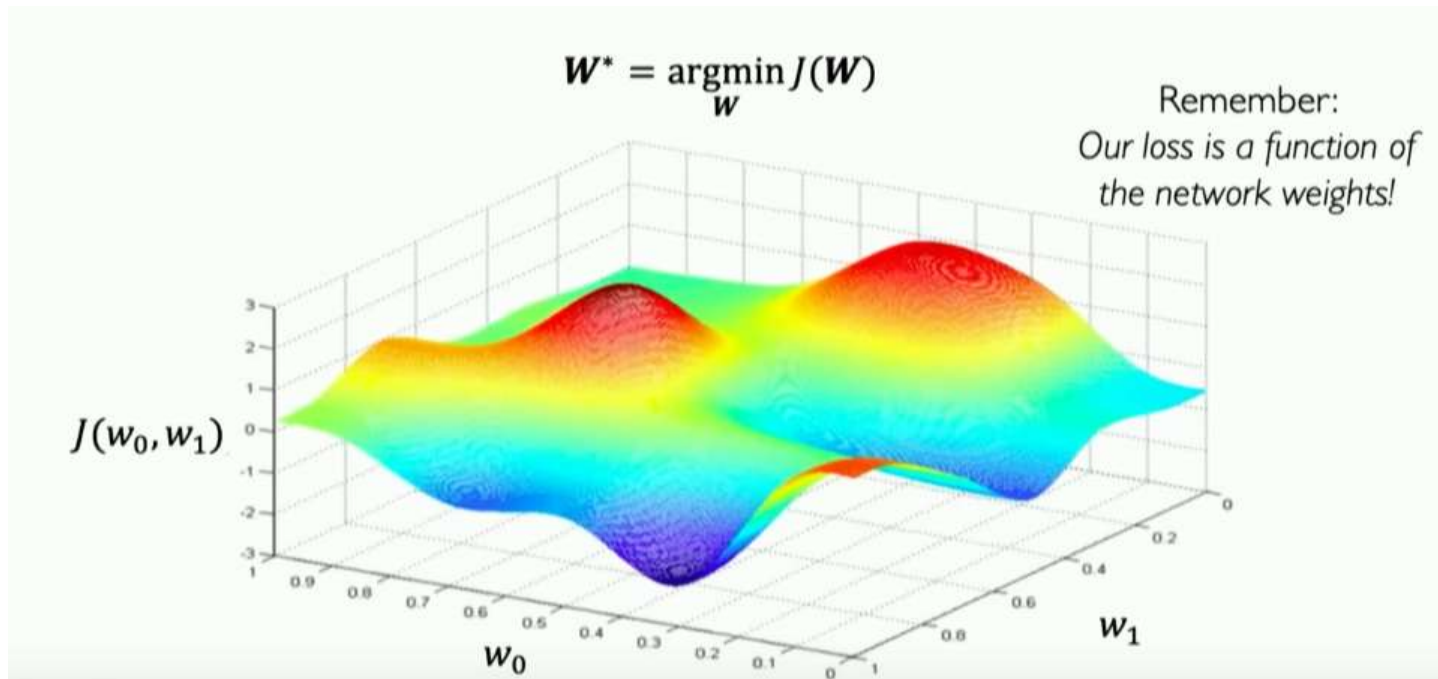
We want to find the network weights that *achieve the lowest loss*

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

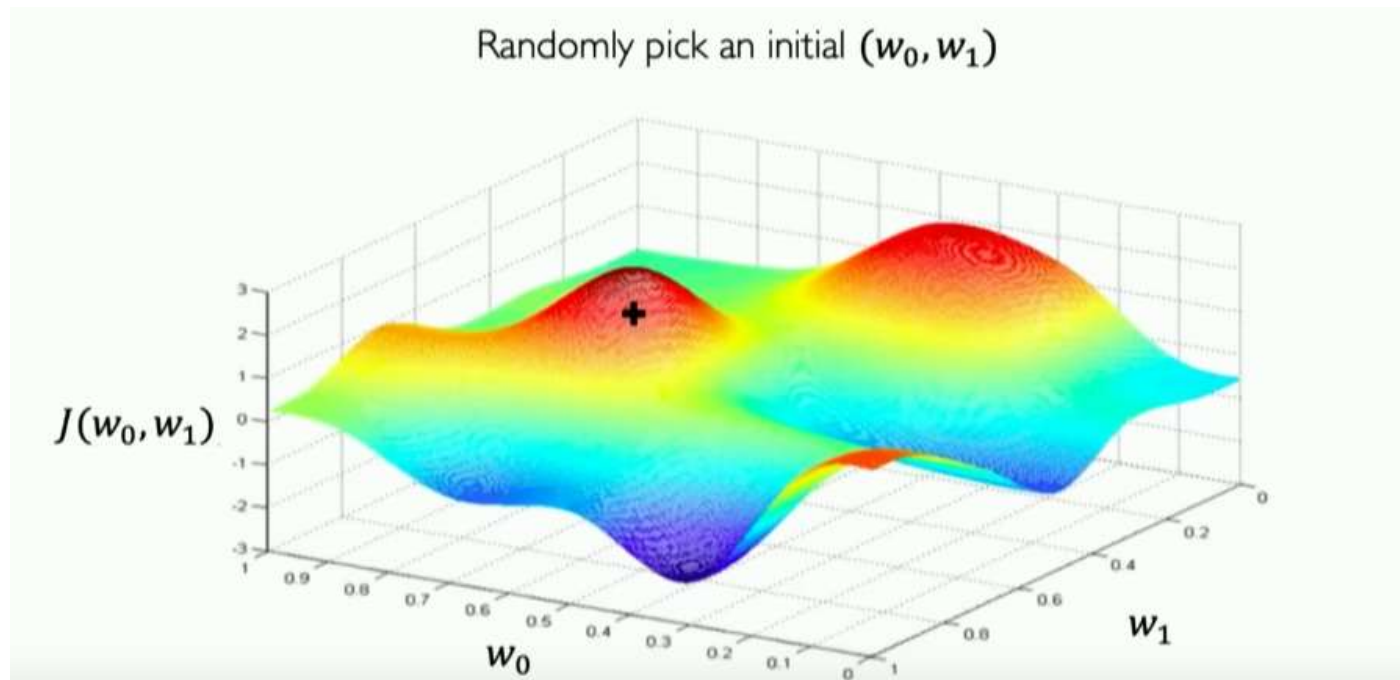
$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

Remember:
 $\mathbf{W} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots\}$

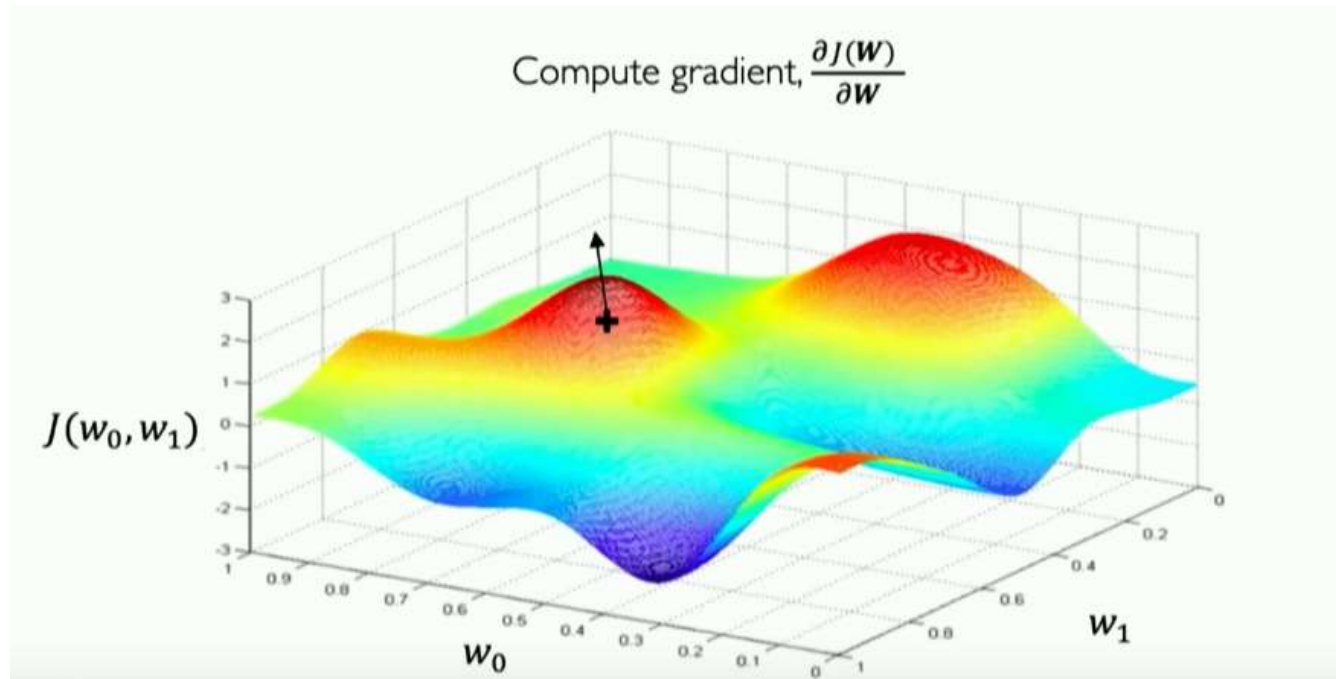
LOSS OPTIMIZATION



LOSS OPTIMIZATION

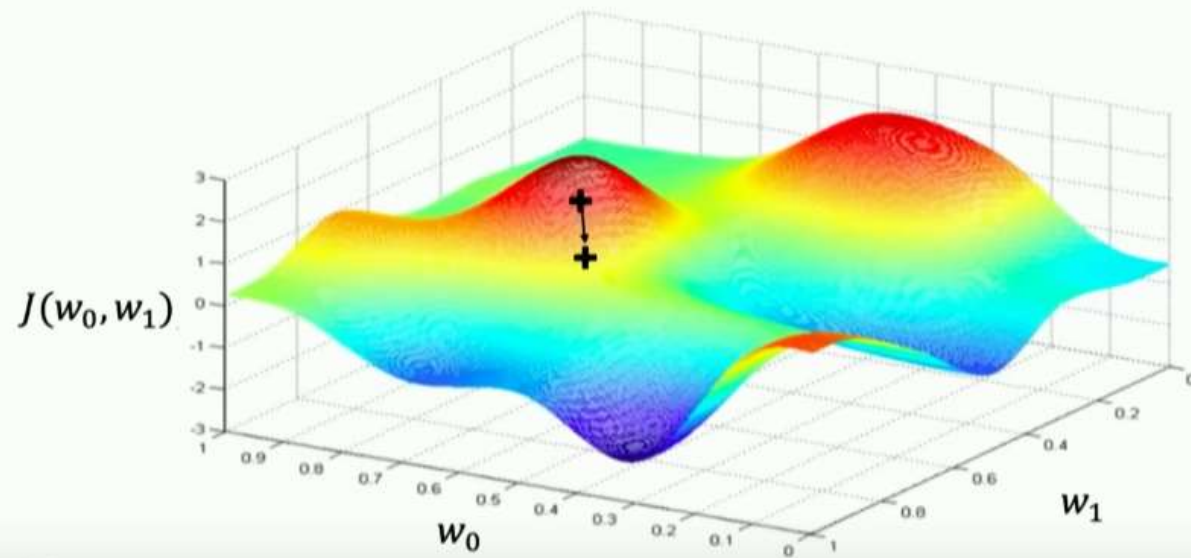


LOSS OPTIMIZATION

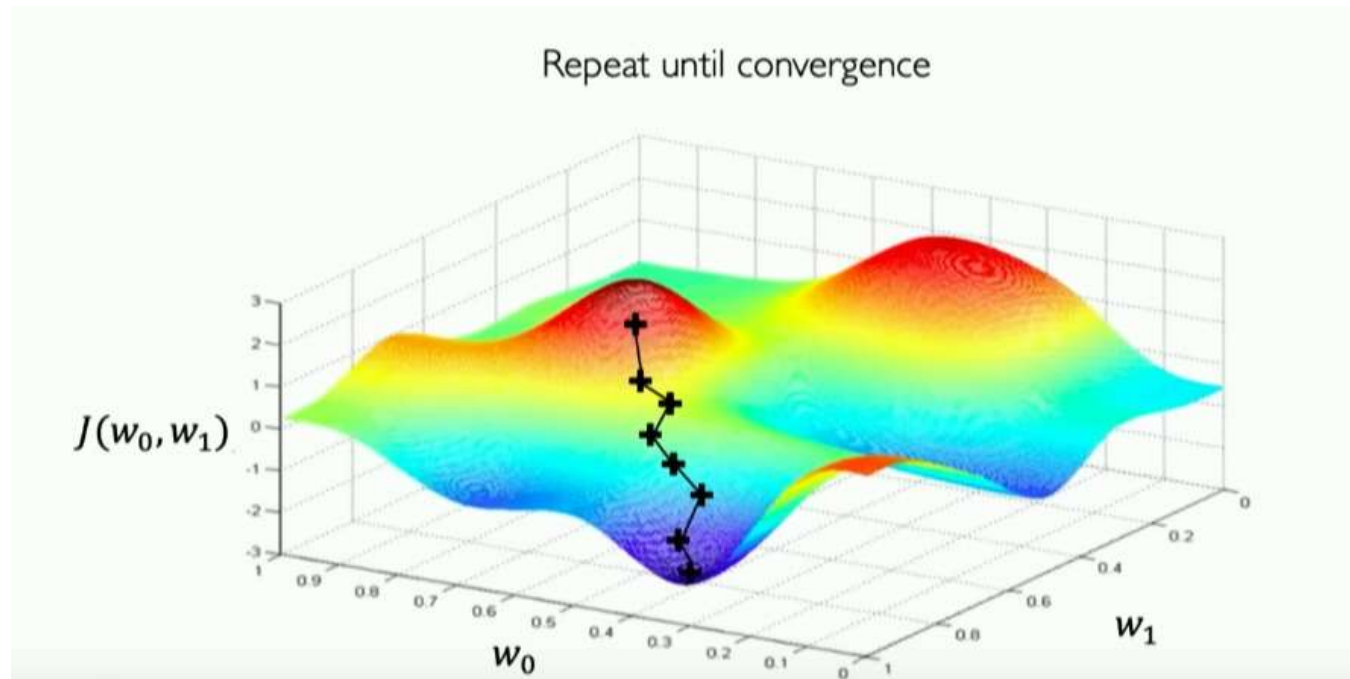


LOSS OPTIMIZATION

Take small step in opposite direction of gradient



GRADIENT DESCENT



GRADIENT DESCENT

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights

GRADIENT DESCENT

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

```
import tensorflow as tf

weights = tf.Variable([tf.random.normal()])

while True:    # loop forever
    with tf.GradientTape() as g:
        loss = compute_loss(weights)
        gradient = g.gradient(loss, weights)

    weights = weights - lr * gradient
```

GRADIENT DESCENT

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

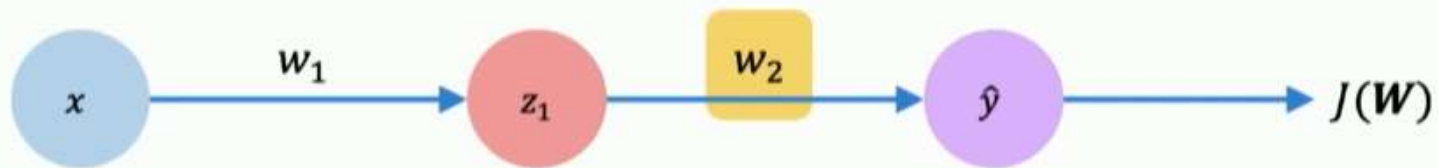
```
import tensorflow as tf

weights = tf.Variable([tf.random.normal()])

while True:    # loop forever
    with tf.GradientTape() as g:
        loss = compute_loss(weights)
        gradient = g.gradient(loss, weights)

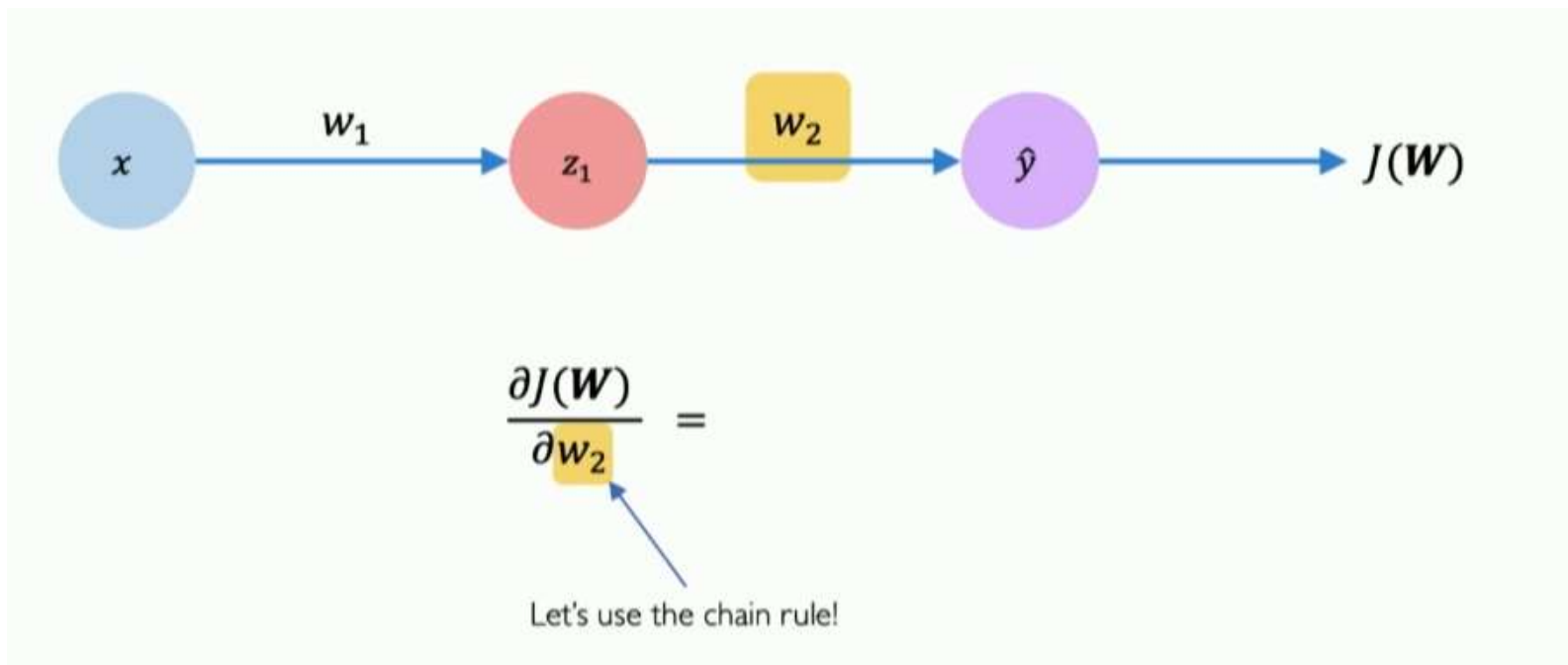
    weights = weights - lr * gradient
```

COMPUTING GRADIENTS: BACKPROPAGATION

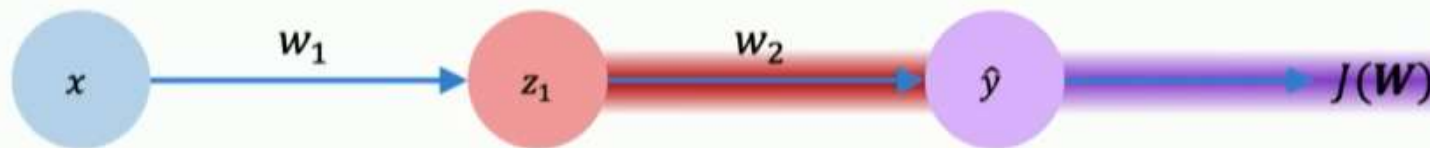


How does a small change in one weight (ex. w_2) affect the final loss $J(\mathbf{w})$?

COMPUTING GRADIENTS: BACKPROPAGATION

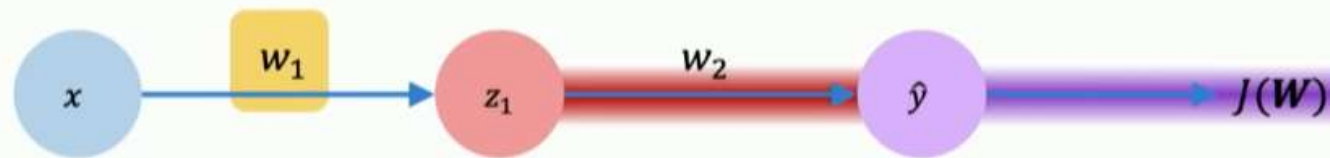


COMPUTING GRADIENTS: BACKPROPAGATION



$$\frac{\partial J(W)}{\partial w_2} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_{\text{red}}$$

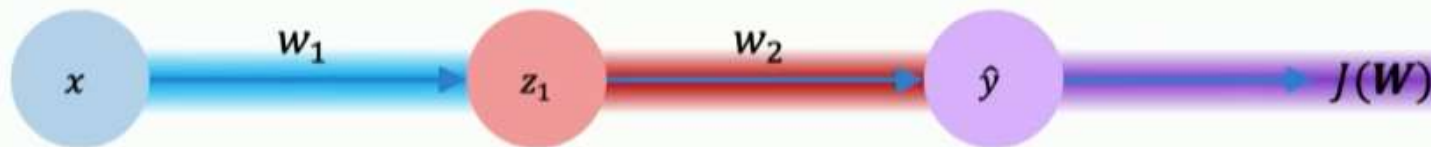
COMPUTING GRADIENTS: BACKPROPAGATION



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

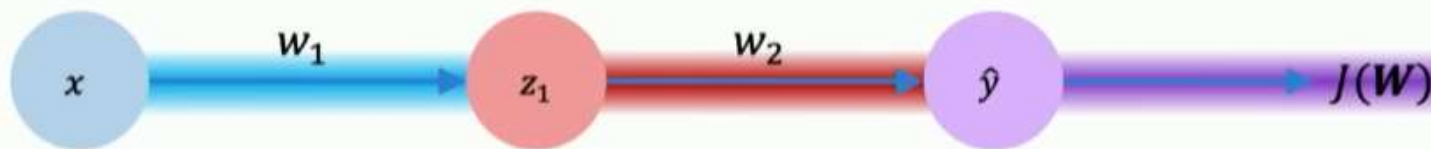
Apply chain rule! Apply chain rule!

COMPUTING GRADIENTS: BACKPROPAGATION



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

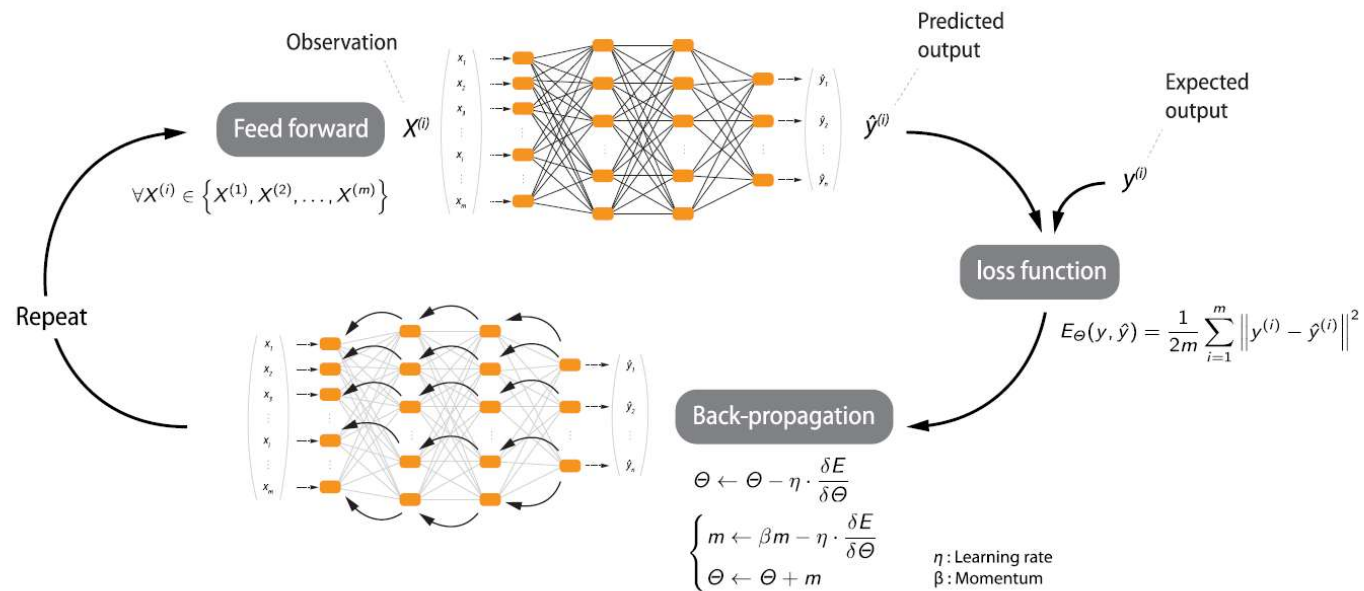
COMPUTING GRADIENTS: BACKPROPAGATION



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

Repeat this for **every weight in the network** using gradients from later layers

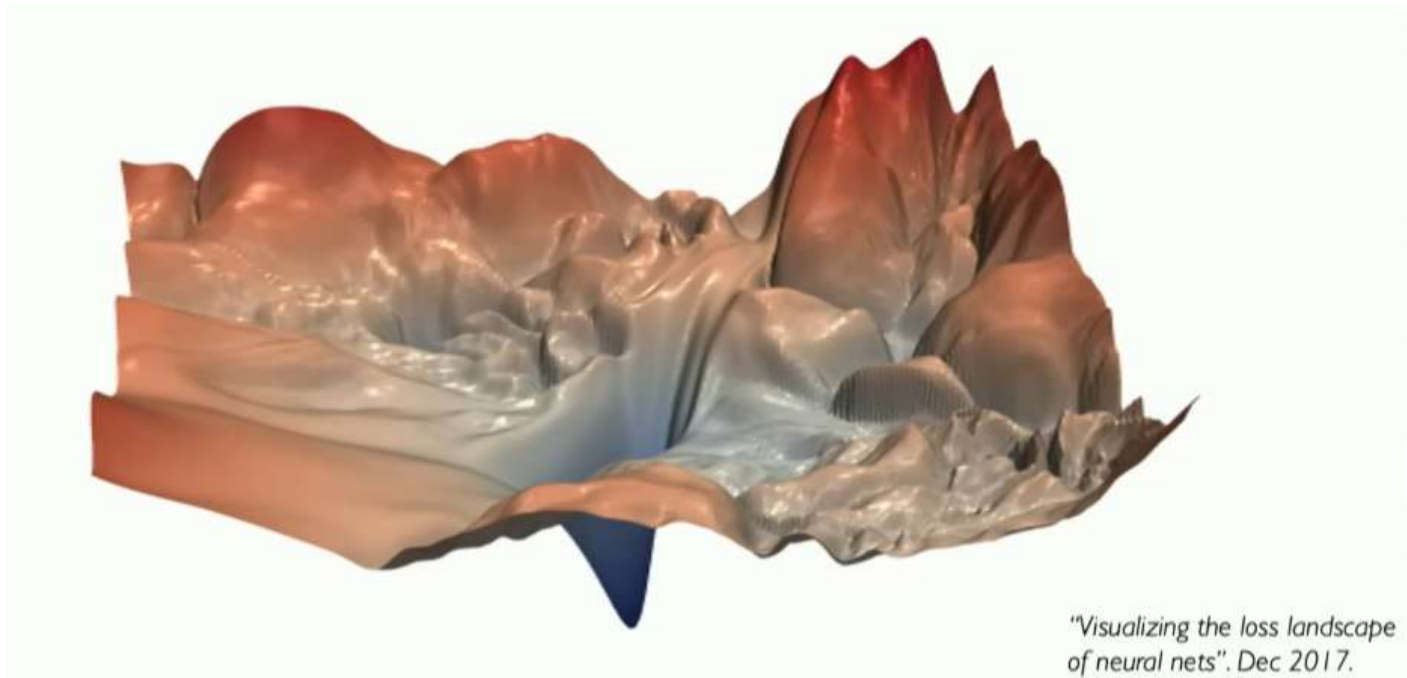
TRAINING PROCESS



NEURAL NETWORKS IN PRACTICE: OPTIMIZATION



TRAINING NEURAL NETWORKS IS DIFFICULT



LOSS FUNCTIONS CAN BE DIFFICULT TO OPTIMIZE

Remember:

Optimization through gradient descent

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$

LOSS FUNCTIONS CAN BE DIFFICULT TO OPTIMIZE

Remember:

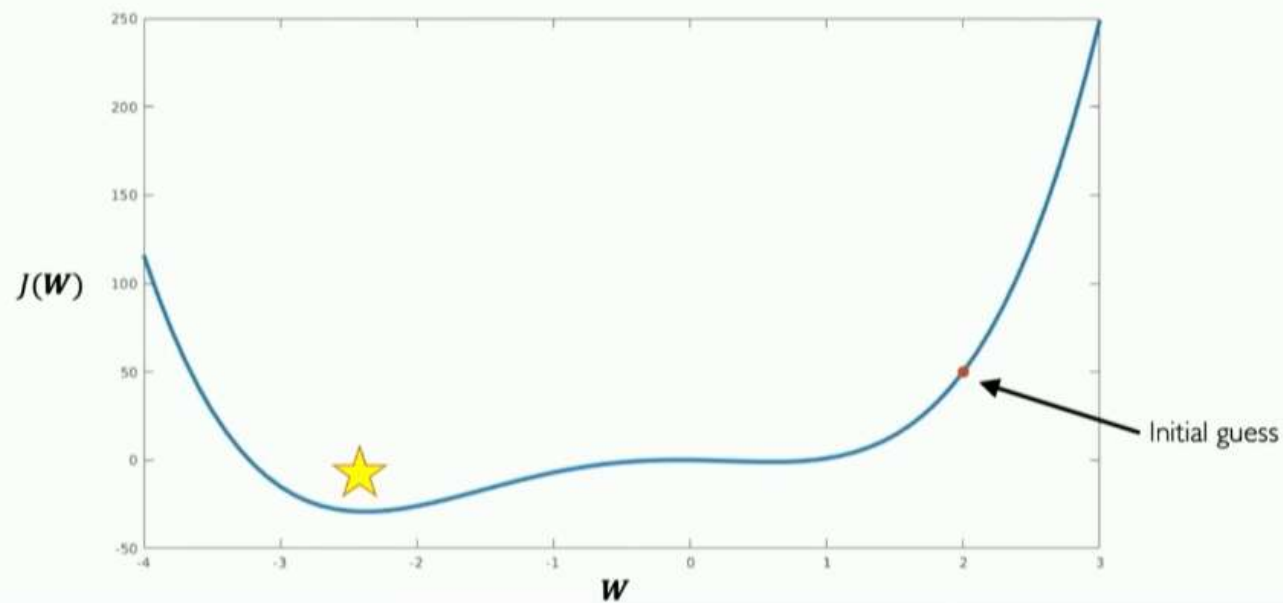
Optimization through gradient descent

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$

How can we set the
learning rate?

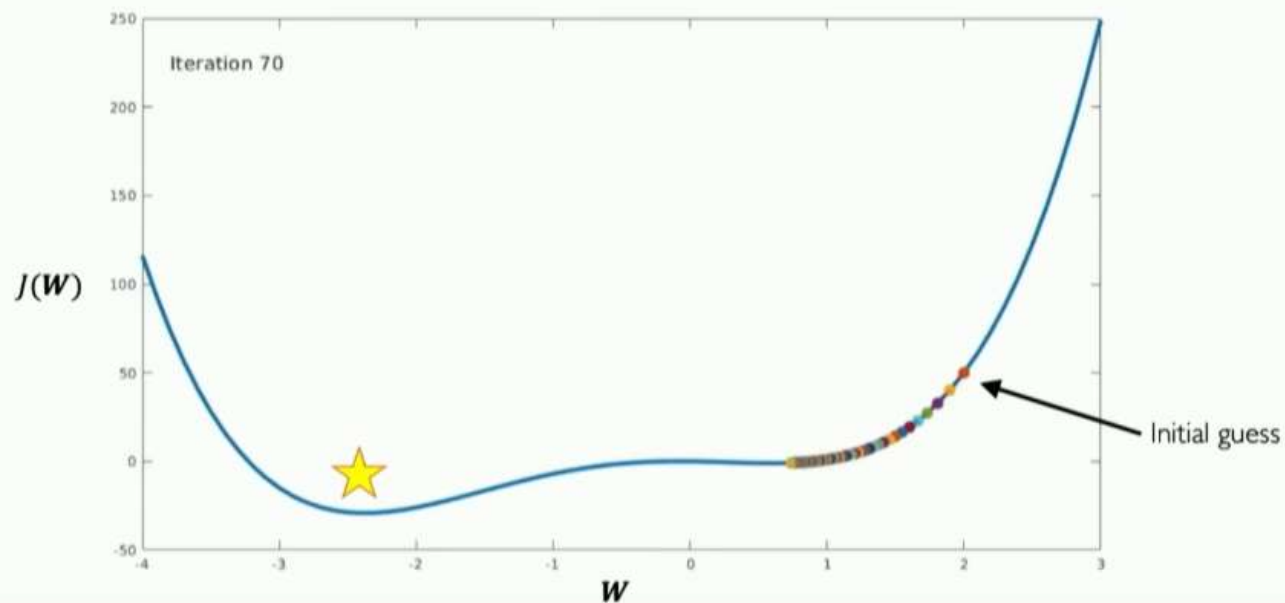
SETTING THE LEARNING RATE

Small learning rate converges slowly and gets stuck in false local minima



SETTING THE LEARNING RATE

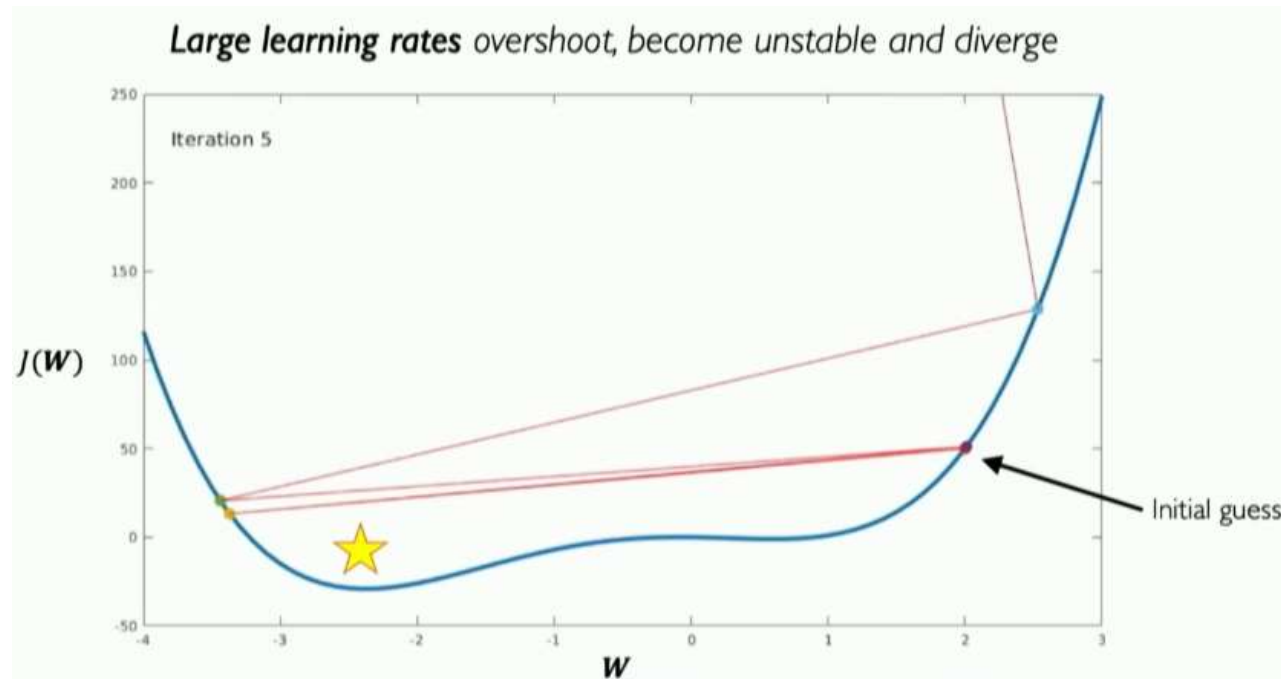
Small learning rate converges slowly and gets stuck in false local minima



SETTING THE LEARNING RATE

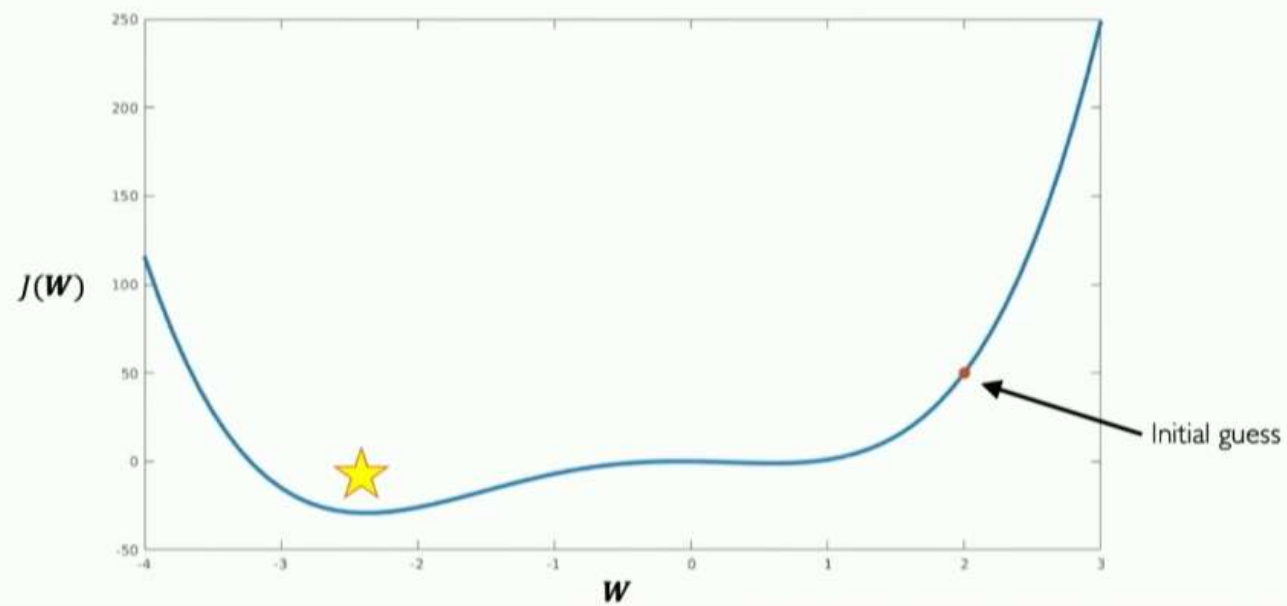


SETTING THE LEARNING RATE

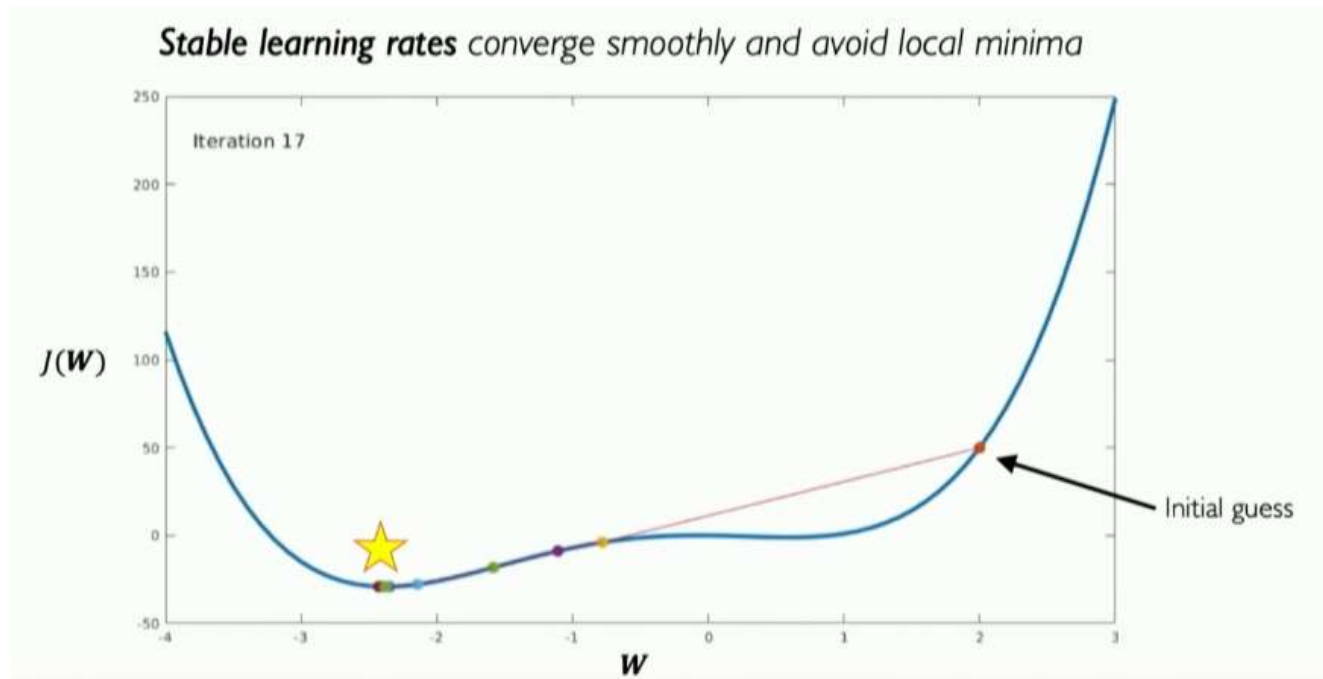


SETTING THE LEARNING RATE

Stable learning rates converge smoothly and avoid local minima



SETTING THE LEARNING RATE



HOW TO DEAL WITH THIS ?

Idea 1:

Try lots of different learning rates and see what works “just right”

HOW TO DEAL WITH THIS ?

Idea 1:

Try lots of different learning rates and see what works "just right"

Idea 2:

Do something smarter!

Design an adaptive learning rate that "adapts" to the landscape

ADAPTIVE LEARNING RATES

- Learning rates are no longer fixed
- Can be made larger or smaller depending on:
 - how large gradient is
 - how fast learning is happening
 - size of particular weights
 - etc...

GRADIENT DESCENT ALGORITHMS

Algorithm

- SGD
- Adam
- Adadelta
- Adagrad
- RMSProp

TF Implementation

	<code>tf.keras.optimizers.SGD</code>
	<code>tf.keras.optimizers.Adam</code>
	<code>tf.keras.optimizers.Adadelta</code>
	<code>tf.keras.optimizers.Adagrad</code>
	<code>tf.keras.optimizers.RMSProp</code>

Reference

Kiefer & Wolfowitz. "Stochastic Estimation of the Maximum of a Regression Function." 1952.

Kingma et al. "Adam: A Method for Stochastic Optimization." 2014.

Zeiler et al. "ADADELTA: An Adaptive Learning Rate Method." 2012.

Duchi et al. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization." 2011.

Additional details: <http://ruder.io/optimizing-gradient-descent/>

PUTTING IT ALL TOGETHER

```
import tensorflow as tf
model = tf.keras.Sequential([...])
# pick your favorite optimizer
optimizer = tf.keras.optimizer.SGD()
while True: # loop forever
    # forward pass through the network
    prediction = model(x)
    with tf.GradientTape() as tape:
        # compute the loss
        loss = compute_loss(y, prediction)
    # update the weights using the gradient
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```



Can replace with any
TensorFlow optimizer!

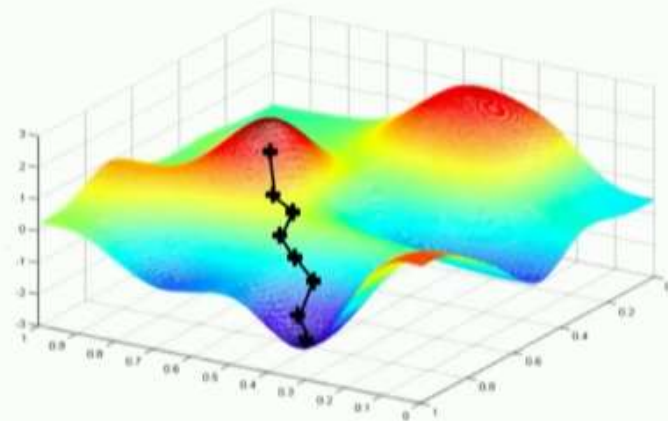
NEURAL NETWORKS IN PRACTICE: MINI-BATCHES



GRADIENT DESCENT

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights

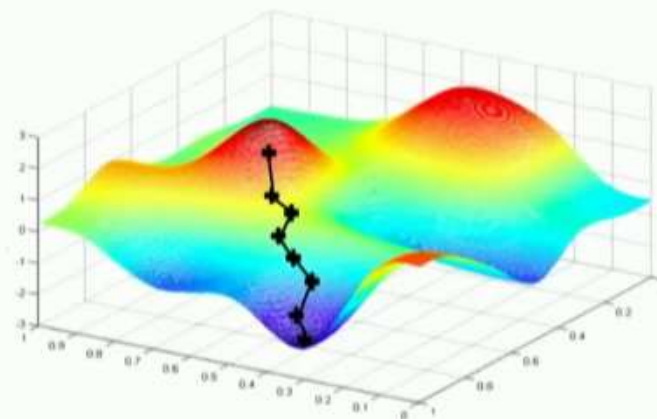


GRADIENT DESCENT

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights

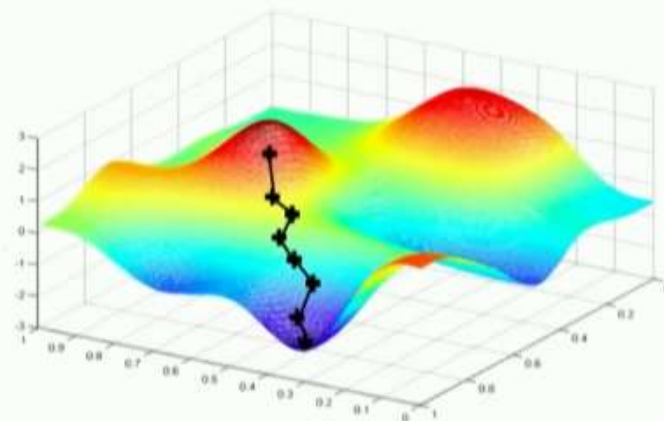
Can be very
computationally
intensive to compute!



STOCHASTIC GRADIENT DESCENT

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point i
4. Compute gradient, $\frac{\partial J_i(\mathbf{w})}{\partial \mathbf{w}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$
6. Return weights

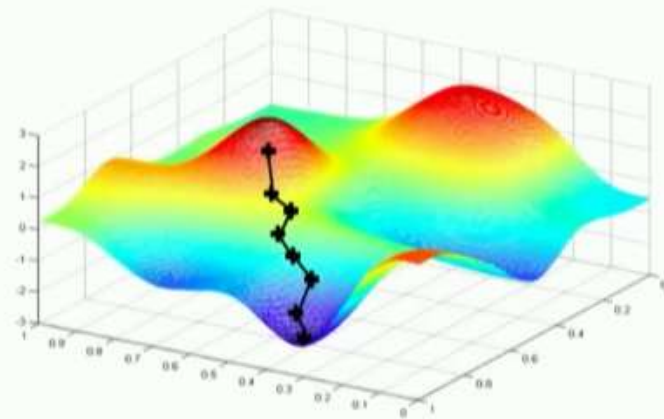


STOCHASTIC GRADIENT DESCENT

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point i
4. Compute gradient, $\frac{\partial J_i(\mathbf{w})}{\partial \mathbf{w}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$
6. Return weights

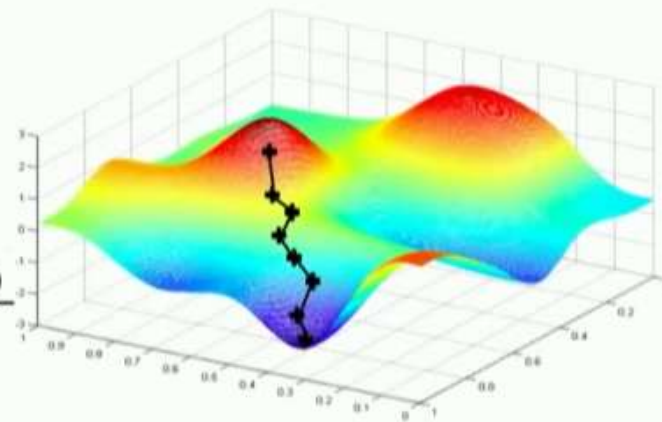
Easy to compute but
very noisy (stochastic)!



STOCHASTIC GRADIENT DESCENT

Algorithm

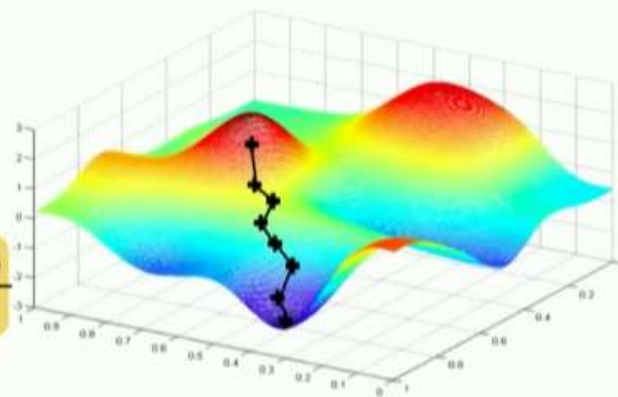
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights



STOCHASTIC GRADIENT DESCENT

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights



Fast to compute and a much better
estimate of the true gradient!

MINI-BATCHES WHILE TRAINING

More accurate estimation of gradient

Smoother convergence

Allows for larger learning rates

MINI-BATCHES WHILE TRAINING

More accurate estimation of gradient

Smoother convergence

Allows for larger learning rates

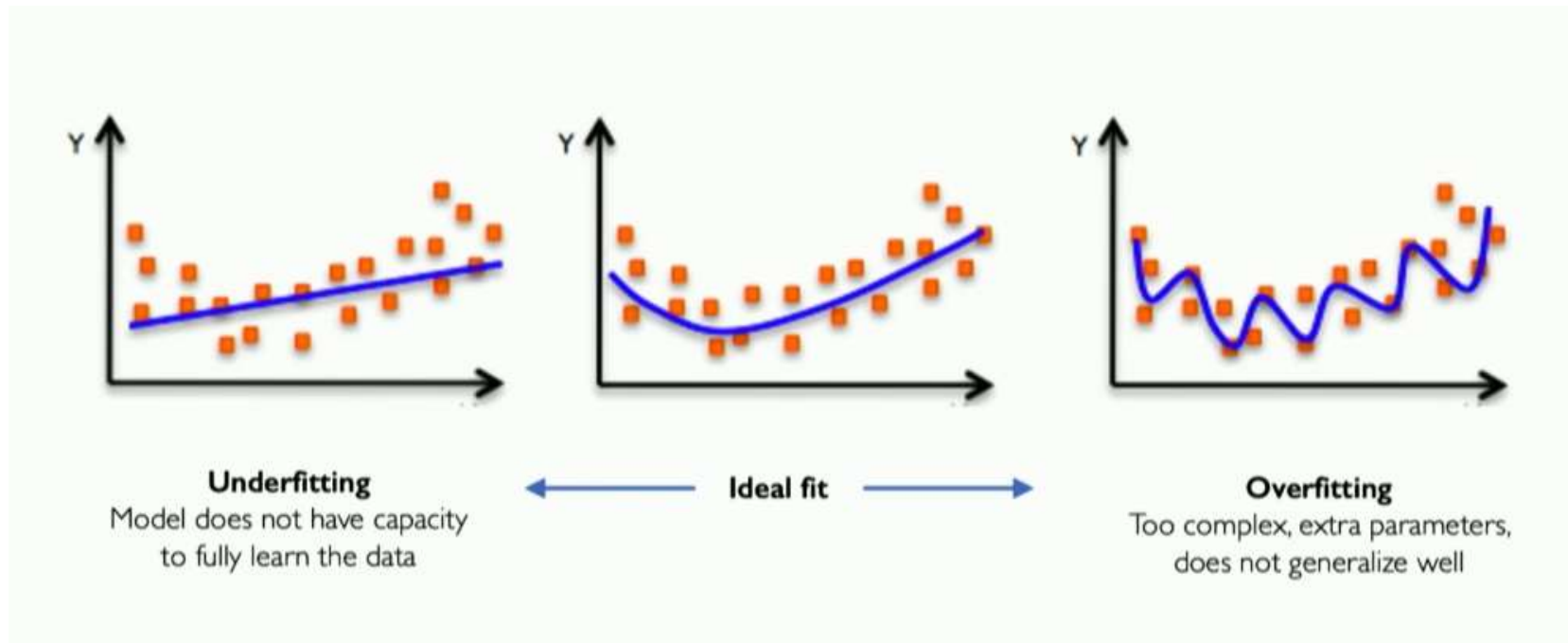
Mini-batches lead to fast training!

Can parallelize computation + achieve significant speed increases on GPU's

NEURAL NETWORKS IN PRACTICE: OVERFITTING



THE PROBLEM OF OVERFITTING



REGULARIZATION

What is it?

Technique that constrains our optimization problem to discourage complex models

REGULARIZATION

What is it?

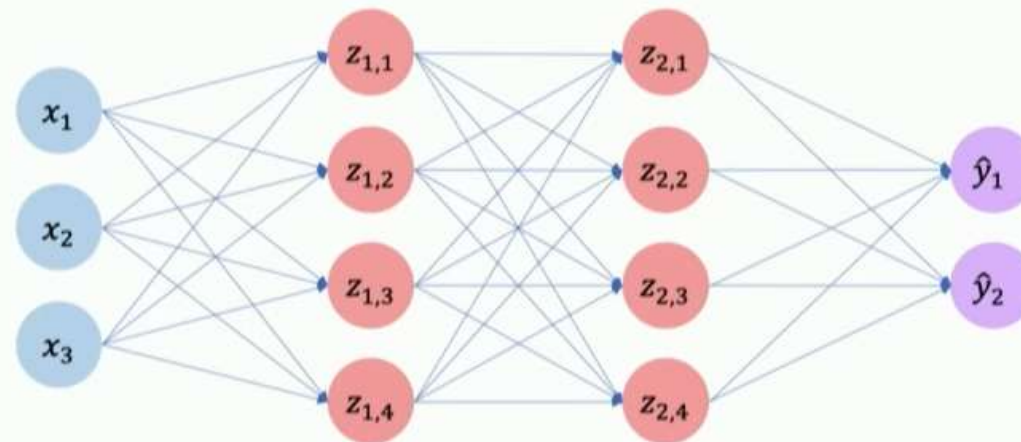
Technique that constrains our optimization problem to discourage complex models

Why do we need it?

Improve generalization of our model on unseen data

REGULARIZATION : DROPOUT

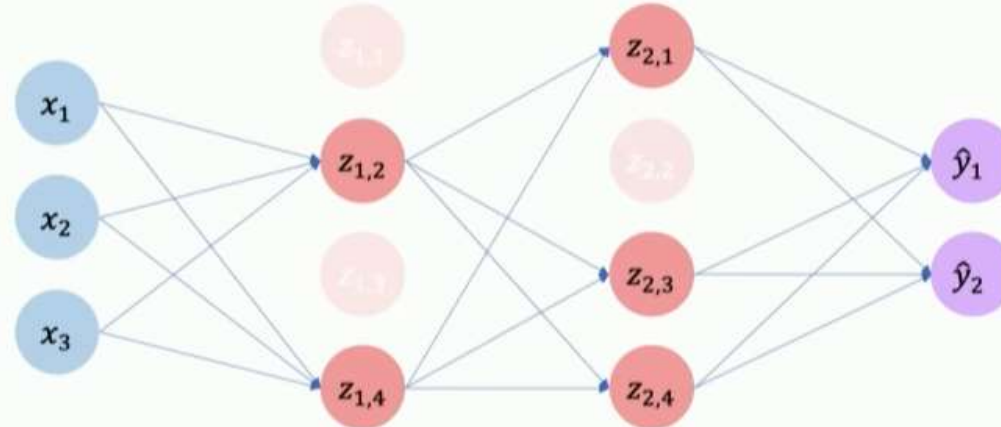
- During training, randomly set some activations to 0



REGULARIZATION : DROPOUT

- During training, randomly set some activations to 0
 - Typically 'drop' 50% of activations in layer
 - Forces network to not rely on any 1 node

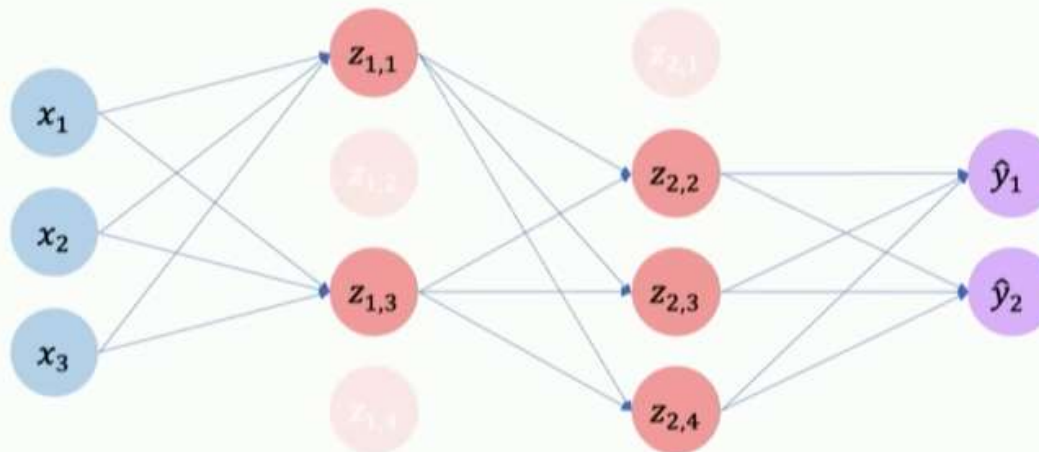
 `tf.keras.layers.Dropout(p=0.5)`



REGULARIZATION : DROPOUT

- During training, randomly set some activations to 0
 - Typically 'drop' 50% of activations in layer
 - Forces network to not rely on any 1 node

 `tf.keras.layers.Dropout(p=0.5)`



REGULARIZATION : EARLY STOPPING

- Stop training before we have a chance to overfit



REGULARIZATION : EARLY STOPPING

- Stop training before we have a chance to overfit



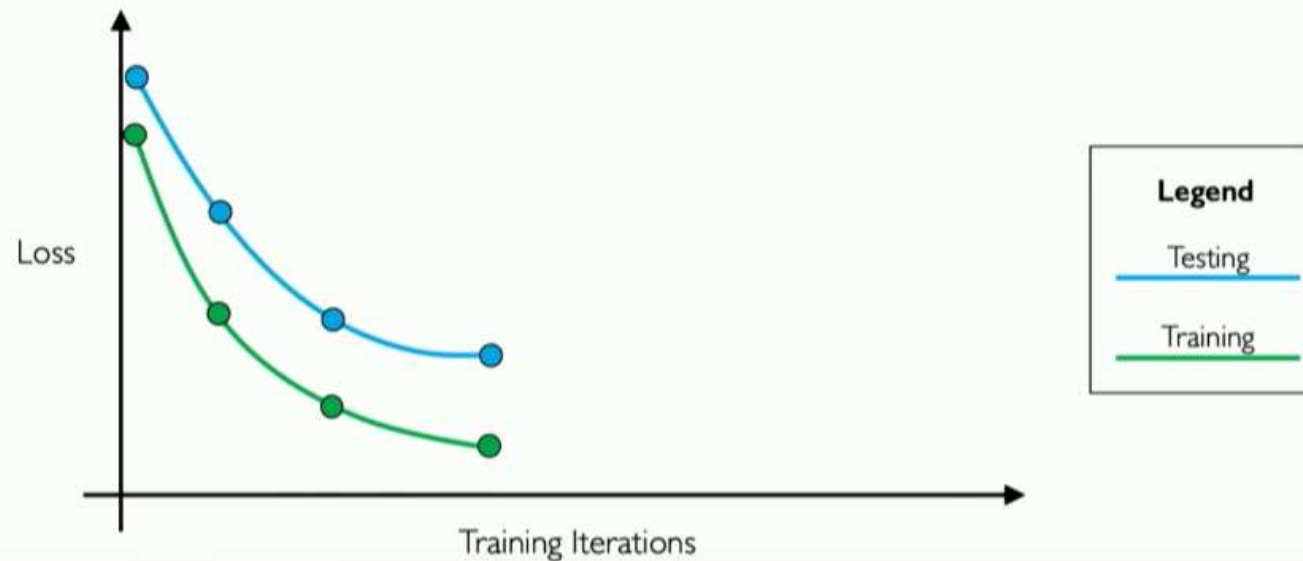
REGULARIZATION : EARLY STOPPING

- Stop training before we have a chance to overfit



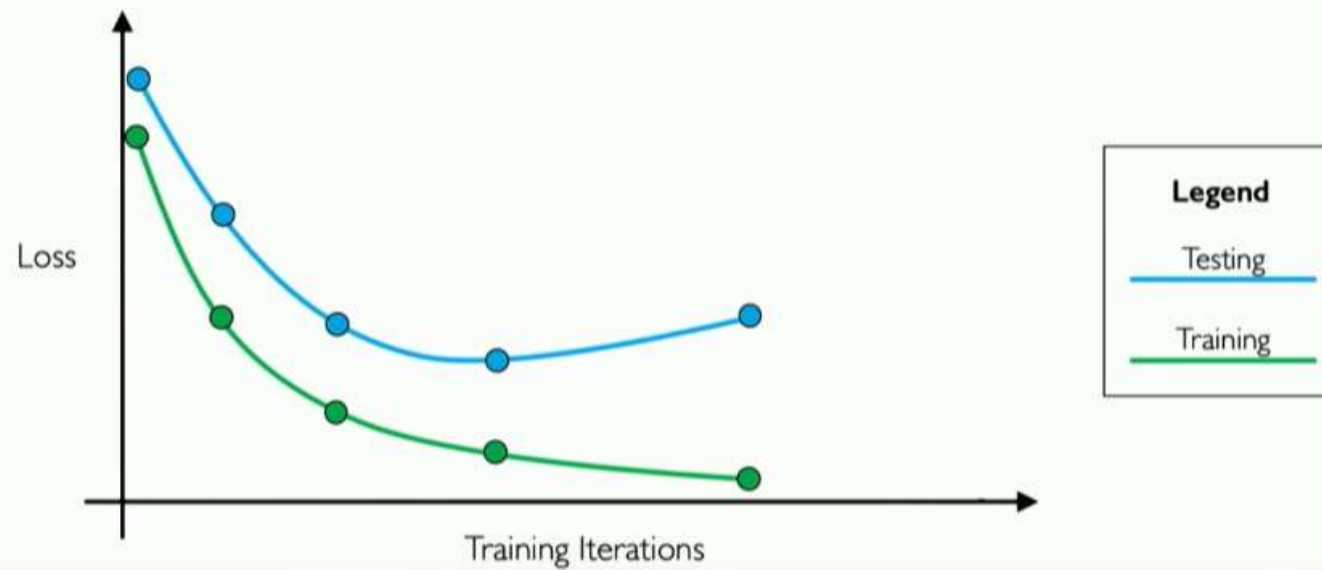
REGULARIZATION : EARLY STOPPING

- Stop training before we have a chance to overfit



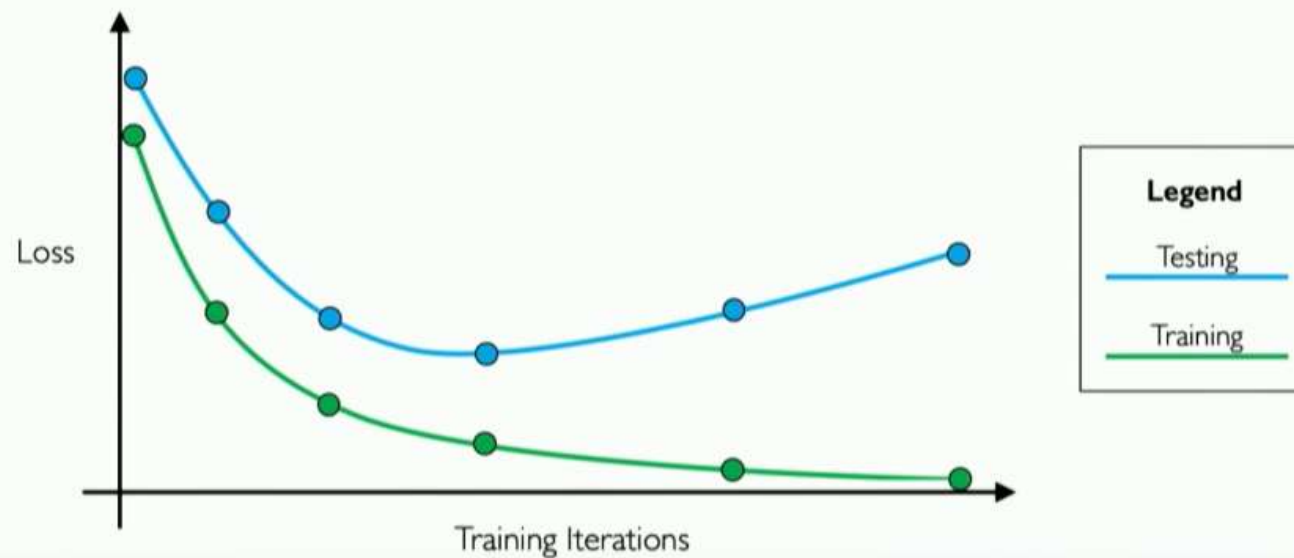
REGULARIZATION : EARLY STOPPING

- Stop training before we have a chance to overfit



REGULARIZATION : EARLY STOPPING

- Stop training before we have a chance to overfit



REGULARIZATION : EARLY STOPPING

- Stop training before we have a chance to overfit



REGULARIZATION : EARLY STOPPING

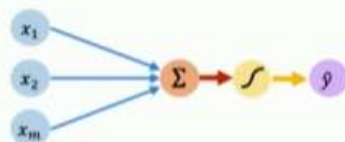
- Stop training before we have a chance to overfit



CORE FOUNDATION REVIEW

The Perceptron

- Structural building blocks
- Nonlinear activation functions



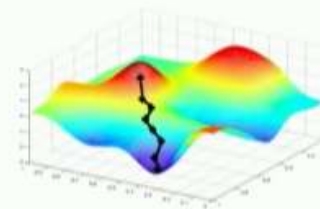
Neural Networks

- Stacking Perceptrons to form neural networks
- Optimization through backpropagation



Training in Practice

- Adaptive learning
- Batching
- Regularization





MERCI